



OPEN

A blockchain-based verifiable CP-ABE scheme for medical data privacy protection

Leilei Zhao^{1,2}, Guofang Dong^{1,2}✉ & Hao Yuan^{1,2}

Electronic Health Record (EHR) sharing in the cloud faces serious challenges, centered on the risk of a single point of failure and privilege abuse due to centralized privilege management, as well as the difficulty of existing mechanisms to effectively validate the integrity of data and access policies in transit/storage. To this end, this paper proposes a scheme that fuses blockchain with verifiable ciphertext policy attribute-based encryption (CP-ABE). Using the decentralized characteristics of blockchain, policy verification is decentralized to multiple nodes, eliminating a single point of failure and enhancing robustness; Vector Commitment technology is introduced to generate a unique binding commitment value for the access policy and incorporate it into the ciphertext to achieve rapid detection of policy tampering and guarantee the authenticity of the enforced policy. Meanwhile, the integrated Attribute-Based Proxy Re-Encryption (AB-PRE) supports dynamic and efficient adjustment of access rights based on patient status and treatment needs. Security analysis and experimental evaluation confirm the efficiency and effectiveness of this scheme.

Keywords Verifiable CP-ABE, Vector commitment, AB-PRE, Access control, Blockchain

With the in-depth development of healthcare informatization, EHR has become a core component of the global healthcare system due to its convenience and efficiency, extensively recording patients' personal information, diagnostic records, genetic history and other highly sensitive data¹. The huge volume of EHR data imposes tremendous pressure on local infrastructure, driving the accelerated migration of healthcare data to the cloud. However, cloud storage exposes EHRs to severe security and privacy challenges: semi-trusted cloud service providers, malicious external attacks, and insiders' unauthorized access constitute major threats of data leakage. The contradiction between these risks and the demand for healthcare data sharing exacerbates the technical complexity of cloud-based EHR systems in terms of data integrity, fine-grained access control, and privacy protection.

In the context of the new era of Internet of Things (IoT) and Artificial Intelligence (AI) convergence^{2–5}, EHR security protection faces particularly high challenges. Attribute-based encryption (ABE) technology, especially CP-ABE^{6–10}, provides an effective approach to realize fine-grained access control based on attributes, with its characteristic of embedding access policies into ciphertexts aligning with EHR scenario requirements. Blockchain technology has also been introduced into the healthcare cloud environment to enhance security via its tamper-proof, transparent, and distributed consensus features¹¹.

Nevertheless, the existing schemes still have certain limitations: (1) Centralized authority management poses prominent risks: most schemes have a potential single point of failure, and misuse or failure of central node authority can lead to system paralysis or sensitive data leakage. The distributed attribute-based encryption (BDABE) scheme proposed by Bramm et al.¹² stores keys in the blockchain but still relies on a centralized authorization model, leading to a single point of failure, computational inefficiency, and lack of scalability; (2) Integrity verification mechanisms lag: existing methods struggle to efficiently and timely verify whether data and access policies have been tampered with during transmission or storage, failing to resist malicious modifications during policy enforcement. The verifiable and traceable ABE scheme by Li et al.¹³ achieves constant ciphertext length and decryption time but fails to address data integrity verification; (3) Dynamic access control is inefficient: when patient conditions or treatment stages change, traditional CP-ABE schemes require full key updates and data re-encryption, leading to high computational and storage overheads, which struggle to adapt to the flexible operational demands of actual medical scenarios.

¹School of Electrical and Information Technology, Yunnan Minzu University, Kunming 650504, China. ²Yunnan Key Laboratory of Unmanned Autonomous System, Yunnan Minzu University, Kunming 650504, China. ✉email: Twzcheima@163.com

Aiming at the challenges of EHR sharing schemes centralization risks, insufficient integrity verification, and inefficient dynamic access control this paper proposes a verifiable CP-ABE scheme integrating blockchain and vector commitments. The scheme leverages the decentralized architecture of blockchain to decentralize policy validation and eliminate single points of failure; it binds access policies with vector commitment technology to enable rapid tampering detection and ensure policy integrity; and it integrates the Attribute-Based Proxy Re-Encryption (AB-PRE) mechanism to achieve dynamic and efficient access right conversion. Committed to promoting secure sharing and efficient utilization of medical data, this solution strictly guarantees data confidentiality and integrity. Its contributions are summarized as follows:

1. Aiming at the problems of insufficient effective verification and lacking fine-grained access control, we design a verifiable CP-ABE mechanism. This mechanism achieves fine-grained access control and reliable verification through attribute encryption and a vector commitment mechanism.

2. Aiming at the inefficiency of dynamic privilege changes that require data owners' participation, this paper proposes an AB-PRE mechanism. The mechanism generates re-encryption keys via a KGC to enable attribute dynamic adjustment without data owners' intervention. It supports privilege changes such as attribute addition, deletion, and modification, with agent nodes undertaking the computational load to improve efficiency. Security of privilege operations is ensured by leveraging the master key and user private keys.

3. Theoretical analysis and security proof show that the proposed scheme can achieve indistinguishability of EHR ciphertexts under chosen plaintext attacks (CPA). Experimental results demonstrate that it effectively resists replay attacks and data tampering, with low computational and communication costs and strong security and high efficiency.

The remainder of this article is structured as follows: Section II reviews related work on electronic health record sharing. Section III introduces the preliminary concepts underlying our proposed scheme. Section IV presents the algorithm design and defines the security model. Section V provides a detailed description of the proposed scheme's construction. Section VI conducts a security analysis. Section VII presents the performance evaluation results. Finally, Section VIII summarizes our scheme.

Related works

Researchers have proposed a variety of technical solutions to address the above challenges in EHR secure sharing, but there are still certain limitations.

Addressing Centralized Authority Risks: In addressing the risk of centralized privilege management, the traditional CP-ABE scheme relying on a single authority (such as the centralized key generation center, KGC) is prone to single-point failure and potential privilege abuse. Blockchain technology is introduced to decentralize trust due to its decentralized nature and tamper-resistance. Ren et al.¹⁴ leverage distributed key management systems (KMS) and zero-knowledge proofs to develop a blockchain-based CP-ABE scheme, aiming to eliminate dependency on centralized authority. The Advance-MedRec scheme by Yang et al.¹⁵ and the approaches proposed by Guo et al.¹⁶ both utilize blockchain to record access events or manage keys. Additionally, Yang et al.¹⁷ enhance the operability and privacy protection of access control through Ethereum smart contracts in their Ancile framework, while Yu et al.¹⁸ propose a blockchain-enabled dynamic attribute update scheme for Internet of Things (IoT) systems. However, these schemes either still require centralized components for critical operations or fail to effectively verify the integrity of the access policy itself, leaving centralization risks incompletely mitigated.

Ensuring Data and Policy Integrity: In terms of safeguarding data and access policy integrity, existing research predominantly focuses on data validation, traceability, or specific query functions^{19–27}. The schemes proposed by Sethi et al.¹⁹ and Yang et al.²⁰ emphasize verifiability and traceability, yet they mainly target user attributes or signatures, falling short of an efficient mechanism to check the tampering of the core access policy embedded in ciphertexts. The ABSE scheme proposed by Liu et al.²¹ necessitates the assistance of trusted organizations for each verification, resulting in inefficiency and the introduction of extra trust points. Meanwhile, the TCKS scheme proposed by Miao et al.²² centers on time-controllable keyword search access control, offering specialized integrity guarantees for time-constrained and keyword search operations at the access control layer. Vector commitment technology has provided new perspectives for efficient verification. Chu et al.²³ developed the BBVC blockchain transaction verification scheme based on vector commitment, leveraging vector commitment algorithms, double-layer aggregation, and version control to mitigate high blockchain storage costs, but it suffers from poor scalability and high computational overhead. Srinivasan et al.²⁴ introduced the first Hyperproof vector commitment scheme, which enables efficient maintenance and aggregation with homomorphic properties to enhance the verification efficiency of stateless cryptocurrencies, yet it contends with issues such as large public parameters, slow verification, and opacity. Campanelli et al.²⁵ proposed Incrementally Aggregatable Vector Commitments (SVC), though the scheme is plagued by excessive computational complexity. Tomescu et al.²⁶ put forward an efficient stateless cryptocurrency scheme based on Aggregatable Subvector Commitments (aSVC), but its anti-attack mechanism for dynamic user updates remains imperfect. Overall, these schemes either grapple with high storage or computation overhead, large public parameters, or slow verification speeds, or they lack optimization for ABE policy structures, rendering them challenging to directly and efficiently verify the integrity of access policies in CP-ABE ciphertexts.

Enabling Efficient Dynamic Access Control: In terms of realizing efficient dynamic privilege management, changes in patient status and treatment needs in EHR scenarios necessitate flexible adjustment of access privileges, with existing studies primarily relying on Proxy Re-Encryption (PRE) techniques and attribute revocation mechanisms^{28–33}. Doshi et al.²⁸ proposed a constant length CP-ABE scheme based on PRE, decoupling ciphertext from attributes to enable constant length ciphertext updates, ensuring anti-collusion security under the DBDH assumption, and providing technical support for dynamic privilege management in EHRs. Xiang et al.²⁹ developed an extensible CP-ABE scheme using version-control techniques to support real-time, fine-

grained attribute revocation. Edemacu et al.³⁰ introduced an Ordered Binary Decision Diagram (OBDD) access structure to enhance policy expressiveness while defending against collusion attacks in collaborative e-health data sharing systems. Yang et al.³¹ designed a lightweight m-health system for fine-grained access control of encrypted Electronic Medical Records (EMRs), featuring malicious user tracking and user revocation capabilities. However, these schemes face significant challenges in dynamic privilege adjustment: traditional PRE or revocation-based updates often require data owners to re-encrypt entire datasets or users to update keys, leading to substantial computational and communication overheads. The PSDQ scheme by Miao et al.³⁴ focuses on privacy protection for dynamic spatial data queries, while the PAFR-ABE scheme by Liang et al.³⁵ integrates Designated Validator Signature (DVS) and Chinese Residual Theorem (CRT) for key revocation and recovery. Nevertheless, such schemes struggle to meet the real-time demands of high-frequency, fine-grained policy-level privilege adjustments in EHR systems.

Although existing research has made certain progress in addressing the challenges of EHR sharing, several significant limitations remain. Firstly, most schemes fail to completely eliminate the risks associated with centralized authentication, leaving systems vulnerable to single - point failures and potential privilege abuse. Secondly, there is a lack of efficient authentication mechanisms to verify the integrity of access policies, especially those embedded in ciphertexts. Thirdly, dynamic privilege adjustment strategies often encounter efficiency bottlenecks and lack generality, particularly when dealing with policy - level changes. Collectively, these limitations severely impede the practical implementation of related technologies in complex, high - security healthcare scenarios.

In summary, while existing blockchain-based verifiable CP-ABE studies for EHR systems cover multiple dimensions, optimization is needed. This paper proposes a blockchain-integrated verifiable CP-ABE scheme for healthcare data privacy protection, aiming to enhance security and execution efficiency by enabling efficient access policy verification and dynamic attribute updates, addressing the aforementioned technical challenges.

Preliminary

In this section, we outline the cryptographic primitives employed in our scheme, including bilinear pairs, mathematical hard problems (discrete logarithm problem, DLP; decisional bilinear Diffie-Hellman problem, DBDH), and vector commitments.

Bilinear maps

A bilinear pair is a mapping between two elliptic curve groups that exhibits specific mathematical properties, such as bilinearity and non-degeneracy. Suppose $\mathbb{G}_1$ and $\mathbb{G}_2$ are multiplicative cyclic groups of order p , where p is a large prime number. Let g be a randomly selected element from the group $\mathbb{G}_1$. A bilinear pair e , specified as $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$, possesses the following characteristic:

- (1) Bilinearity: For any $P, Q \in \mathbb{G}_1$ and $a, b \in \mathbb{Z}_p^*$, there is $e(P^a, Q^b) = e(P, Q)^{ab}$.
- (2) Non-degeneracy: Let g be a generating element of $\mathbb{G}_1$, then $e(g, g) \neq 1$.
- (3) Computability: For $\forall P, Q \in \mathbb{G}_1$, $e(P, Q)$ can be efficiently computed.

Difficult assumption

(a) Discrete Logarithm Problem(DLP)

Discrete Logarithm Problem (DLP)³⁶: Consider a cyclic multiplicative group $\mathbb{G}$ with prime order p , and let g be a generator of $\mathbb{G}$. The DLP is defined as follows: given g and $g, y \in \mathbb{G}$, determine the exponent $a \in \mathbb{Z}_p^*$ such that $y = g^a$. From the standpoint of computational complexity theory, for any probabilistic polynomial time algorithm A , the probability of successfully solving the DLP is negligible. In other words (Eq. 1)

$$\text{advDLP}_A = \Pr[A(g, g^a) = a, a \in \mathbb{Z}_p^*] \leq \varepsilon \quad (1)$$

where ε is an insignificant probability value.

(b) Decisional Bilinear Diffie-Hellman Problem(DBDH)

Decisional Bilinear Diffie-Hellman Problem (DBDH)³⁷: Given bilinear groups $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$, as well as bilinear pairs $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, generators $P \in \mathbb{G}_1$, and randomly chosen integers $a, b, c \in \mathbb{Z}_p^*$, and determine whether $e(P, P)^{abc}$ is equal to $e(P, P)^Z$. DBDH is hard if $e(P, P)^{abc} = e(P, P)^Z$, which is hard for all PPT opponents A .

Vector commitment

Vector Commitment (VC) is a cryptographic tool³⁸. A vector commitment scheme usually consists of an initialization algorithm, *Setup*, a commitment algorithm, *Com*, an opening algorithm, *Open*, and a verification algorithm, *Ver*, which implements vectors' commitment, opening, and verification functions. The formal definition is given below:

Setup ($1^\lambda, d$) $\rightarrow (vp, cp)$: Take the security parameter λ and the vector dimension d as inputs and output the public parameter cp and the verification parameter vp needed for the scheme.

Com (cp, v) $\rightarrow (C_{co}, st)$: The prover takes the public parameter cp and a d -dimensional vector v over the number field F as input. It outputs the commitment C_{co} and the state of the committer st . where the d -dimensional vector $v = \{v_1, v_2, \dots, v_d\}$.

Open ($cp, C_{co}, i \in d, v/st$) $\rightarrow (v_i, \pi_i)$: The prover takes the commitment C_{co} and some specified position i as public inputs, and with the private input v , outputs the value v_i of the committed vector at position i and the corresponding proof π_i .

$Ver(vp, C_{co}, i, v_i, \pi_i) \rightarrow (1/0)$: The verifier verifies using C_{co} , location i , value v_i , and proof π_i . If the value of the vector in commitment C_{co} at position i is confirmed to be v_i , it outputs 1; otherwise, it outputs 0.

Algorithm and security model definitions

In this section, we elaborate on the system model of our scheme, the description of the verifiable CP - ABE scheme, formal definitions, security models, and design goals. Additionally, the specific meanings of the symbols and parameters involved in this scheme are presented, as shown in Table 1.

System model

This article designs a secure and efficient EHR access solution between various departments of hospitals. It focuses on solving the problems of safe and flexible access control and dynamic rights management of EHR among multiple departments within the hospital. The scheme mainly consists of four entities: key generation center, hospital, blockchain, and cloud. The entity interaction system model is presented in Fig. 1.

- Key Generation Center (KGC): KGC primarily generates parameters for each entity and registration keys for all users, serving as the trust anchor that ensures system security.
- Hospitals: as the core of the whole system, they include data owner and data user roles.
 - Data Owner (DO): DOs define access policies and encrypt EHRs via verifiable CP-ABE.
 - Data User (DU): DUs attempt to decrypt the information based on its attribute key. They can only successfully decrypt and retrieve the information if their attributes match the access policy.
- Blockchain (BC): BC serves as a federated chain for recording operation logs, providing an open, transparent and tamper-proof operating environment.
- Cloud Service(CS): CS is used to store encrypted medical record files securely. A semi-trusted cloud environment, with encryption technology and an integrity-checking mechanism, guarantees the high confidentiality and integrity of the data during transmission and storage and reduces the possibility of data leakage or tampering due to the potential risks of cloud services.

We elaborate through a concrete example to better understand the interaction process between the data owner and the data user. Suppose that the Department of Surgery (the data owner) needs to encrypt and upload the patient's EHRs in a hospital environment. In contrast, the Department of Radiology (the data user) must download and decrypt these records to view the relevant information. Figure 2 shows how DOs and DUs interact, with the process detailed below.

(1) Data owner (surgeon general)

The surgeon defines the access policy tree Γ_1 (e.g., "Radiology" AND "Attending Physician") attributes that can be accessed by personnel. The EHR data is encrypted using the system parameter PP and the access policy tree Γ_1 to generate the ciphertext CT . The surgeon proceeds to calculate the policy commitment value $Commit(p)$ and incorporates it into the ciphertext CT . Then, the generated ciphertext CT is uploaded to the cloud server, as well as related metadata, etc., to the blockchain for subsequent verification.

(2) Data user (Radiology)

The radiologist downloads the ciphertext CT from the cloud server, extracts the policy commitment value $Commit(p)$ and the access policy tree Γ_1 from CT , locally recalculates $Commit(p')$, and compares whether the two values are equal. If they are not equal, the ciphertext tampers and the computation is terminated; if they are equal, attribute decryption is performed, and the radiologist decrypts the ciphertext CT using the attribute key to obtain the plaintext M .

Description of the verifiable CP-ABE scheme

Within the verifiable CP-ABE framework, the access policy is modeled as a tree structure where each node maps to a specific attribute. By linking each attribute to a unique position in the commitment vector, the scheme enables only users with the required attributes to reveal the commitment at the corresponding position. The core design of our scheme is detailed as follows:

- (1) $Setup(1^\lambda, d) \rightarrow (cp, vp, MK)$: KGC takes security parameter λ and the vector dimension d as input and output disclosure parameter cp , verification parameter vp and master key MK ;
- (2) $Keygen(cp, MK, S, \Gamma_1) \rightarrow AK_1$: KGC inputs the public parameter cp , the master key MK , the set of user attributes S , and the access policy Γ_1 and outputs the attribute key AK_1 for the corresponding location.
- (3) $Com(cp, \Gamma_1) \rightarrow Commit(p)$: DO inputs the public parameter cp and the access policy tree Γ_1 and outputs the access policy commitment $Commit(p)$.
- (4) $Enc(cp, M, \Gamma_1) \rightarrow CT$: DO takes the public parameter cp , the plaintext M , and the access policy tree Γ_1 as inputs and outputs the ciphertext CT .
- (5) $DecVer(CT, AK_1, Commit(p), i, v_i) \rightarrow (M, \perp)$: DU inputs the ciphertext CT , the attribute key AK_1 , the policy commitment $Commit(p)$, the location i and the value v_i , and outputs the plaintext M or $\perp$.

Formal definition

Our proposed scheme includes fourteen probabilistic polynomial-time algorithms, detailed as follows.

- (1) $Setup(1^\lambda, d) \rightarrow (PP, MK)$: This algorithm is executed by KGC. It takes as input parameter λ and vector dimension d , and outputs system public parameter PP and master key MK .

Parameter	Definition	Parameter	Definition
$\mathbb{G}_1, \mathbb{G}_2$	Group of multiplicative cyclic	M	Plaintext
g	Generator of $\mathbb{G}_1$	$\text{Commit}(p)$	Policy Commitment
H	Hash function	$C, C', (C_y, C'_y)_{y \in \Gamma_1}$	Ciphertext Components
MK	System master key	CT, CT'	The former is the original ciphertext, the latter is the new ciphertext
$h_i, e(g, g)^\alpha$	Public Key Parameters	RK	Re - encryption Key
h_i	Attribute - Vector Mapping	$RK_1, RK_2, (RK_j, RK'_j)_{j \in S}$	Re - encryption Key Components, where S' is the new attribute set
H_1	Policy commitment parameter	$\Delta_i(0)$	Lagrange Interpolation Coefficient
pp	System parameters	P	Bilinear pairing operation on $\mathbb{G}_1$
S	User attributes sets	M_{G_1}	Multiplication operation on $\mathbb{G}_1$
AK_1, AK_2	The former is the original attribute key, and the latter the new one	A_{G_1}	Addition operation on $\mathbb{G}_1$
$D_i \{D_i, D'_i\} \dots$	Attribute key components	E_{G_1}	Exponentiation operation on $\mathbb{G}_1$
Γ_1, Γ_2	Access trees (the former is the original access tree, the latter is the updated access tree)	H_{Z_p}	Hash operation on $\mathbb{Z}_p^*$

Table 1. Parameters and definitions.

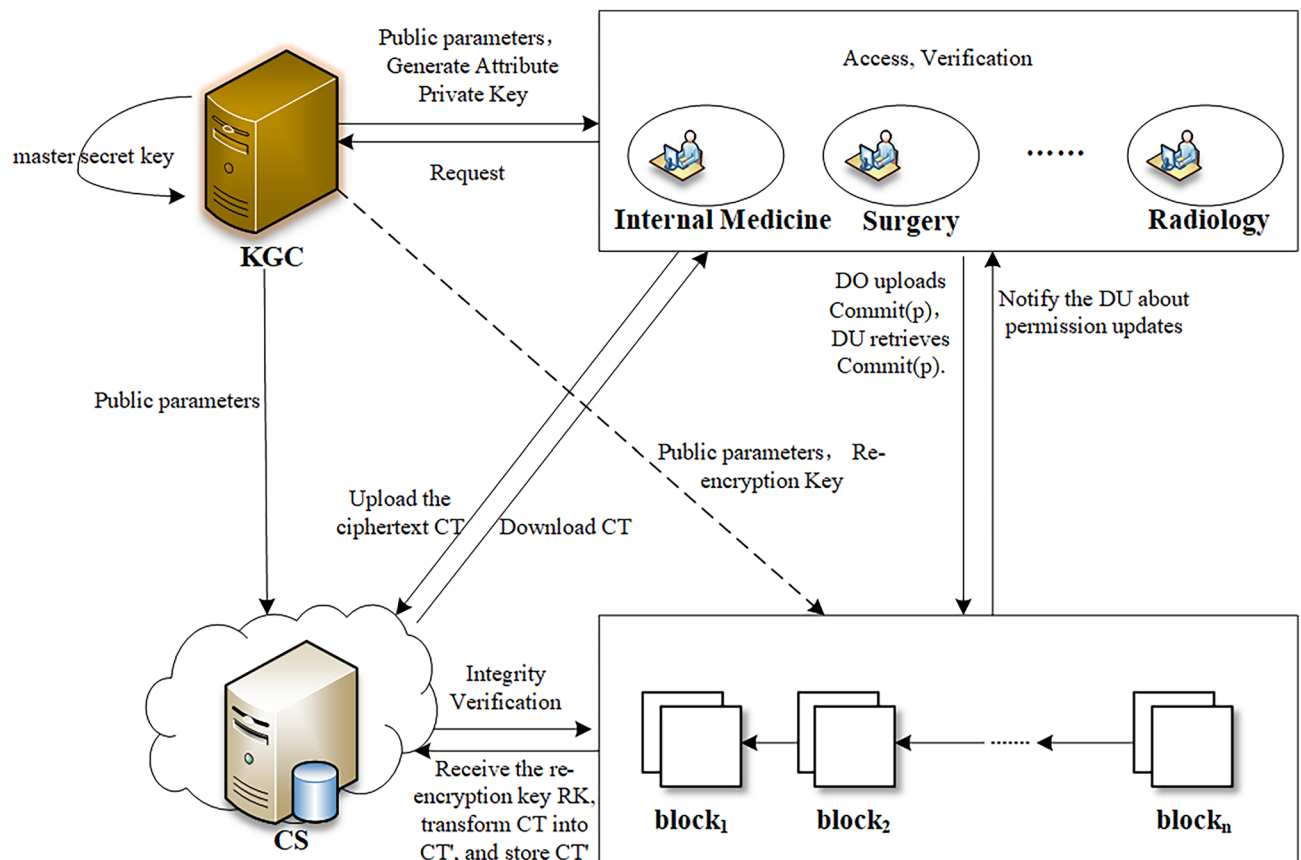


Figure 1. System model diagram.

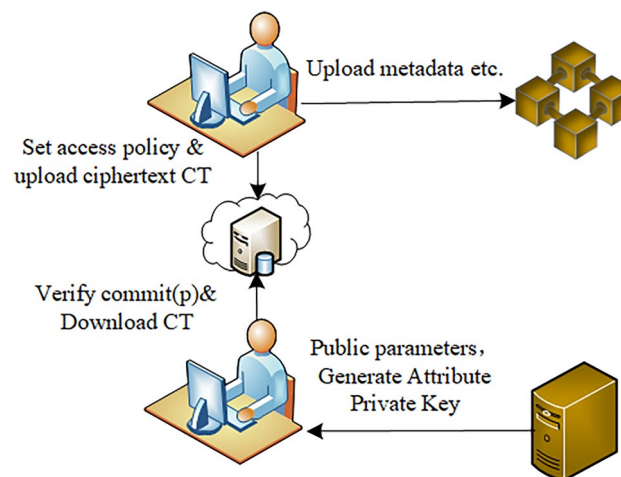


Figure 2. DO and DU interaction diagram.

(2) $KeyGen(PP, MK, S) \rightarrow AK_1$: This algorithm is executed by KGC. It takes as input system parameter PP , system master key MK , and user attribute set S , and outputs attribute key AK_1 .

(3) $TreeGen(\cdot) \rightarrow \Gamma_1$: This algorithm is executed by DO. It takes as input nodes, polynomials, threshold gates, AND/OR branches, etc., and outputs access tree Γ_1 .

(4) $KeyEnc(M, PP) \rightarrow (C, C', C_y, C'_y)$: This algorithm is executed by DO. It takes as input plaintext M and system parameter PP , and outputs ciphertext components C, C', C_y, C'_y .

(5) $PCGen(PP, \Gamma_1) \rightarrow Commit(p)$: This algorithm is executed by DO. It takes as input system parameter PP and access tree Γ_1 , and outputs policy commitment $Commit(p)$.

(6) $CipComUp \left(C, C', \left\{ C_y, C'_y \right\}_{y \in \Gamma_1}, Commit(p) \right) \rightarrow CT$: This algorithm is executed by DO.

It takes as input ciphertext components C, C', C_y, C'_y and policy commitment $Commit(p)$, and outputs ciphertext CT .

(7) $DataAcc(PP, ID) \rightarrow CT$: This algorithm is executed by DU. It takes as input system parameter PP and unique data ID , and outputs ciphertext CT retrieved from the cloud.

(8) $ComVer(CT, PP) \rightarrow Valid/Invalid$: This algorithm is executed by DU. It takes as input system parameter PP and ciphertext CT (containing policy commitment $Commit(p)$) retrieved from the cloud, verifies the validity of the policy commitment, and outputs *Valid* or *Invalid*.

(9) $DataVer(i, PP, v_i, \pi_i) \rightarrow Valid/Invalid$: This algorithm is executed by DU. It takes as input system parameter PP , position i to be verified, value v_i from the VC.Open algorithm, and computation proof π_i , verifies whether v_i is correct at position i , and outputs *Valid* or *Invalid*.

(10) $Dec(PP, CT, AK_1) \rightarrow M/\perp$: This algorithm is executed by DU. It takes as input system parameter PP , ciphertext CT , and attribute key AK_1 , and outputs plaintext M or refuses decryption $\perp$.

(11) $KeyGen_2(PP, AK_1, S', MK) \rightarrow RK$: This algorithm is executed by KGC. It takes as input public parameter PP , original attribute key AK_1 , user's new attribute set S' , and master key MK , and generates re-encryption key RK .

(12) $ReEnc(PP, CT, RK, \Gamma_2) \rightarrow CT'$: This algorithm is executed by CS. It takes as input public parameter PP , ciphertext CT , re-encryption key RK , and new access policy Γ_2 , and generates new ciphertext CT' .

(13) $UpVer(PP, CT', \Gamma_2) \rightarrow 1/0$: This algorithm is executed by DO. It takes the public parameter PP , new ciphertext CT' , and new policy Γ_2 as input, verifies the commitment algorithm, and outputs 1 to indicate ciphertext integrity or 0 to indicate ciphertext tampering.

(14) $PerchaNot(S', PP) \rightarrow AK_2$: This algorithm is executed by KGC. It takes as input new attribute set S' and public parameter PP , and outputs updated new attribute key AK_2 .

Security model

In this article, based on the probabilistic polynomial time interaction of attackers $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ with challenger $\mathcal{B}$, we construct three types of models for policy commitment verification security, privilege change security, and distinguish ability under selective plaintext attack and define the security boundaries through confrontation rules and challenge games. If attackers $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3$ have negligible success probability in each model in polynomial time, the scheme has the corresponding security properties.

Game1 Policy commitment verification security

Initialization: Challenger $\mathcal{B}$ runs the system initialization algorithm and generates the public parameters PP and the master key MK . The challenger $\mathcal{B}$ defines a target policy Γ^* .

Phase1: Attacker $\mathcal{A}_1$ can make adaptive queries under the following polynomial number:

(1)Key generation queries:

Attacker $\mathcal{A}_1$ adaptively initiates a query to challenger $\mathcal{B}$ for access policy Γ . Challenger $\mathcal{B}$ generates the corresponding attribute key AK_1^* and returns it to attacker $\mathcal{A}_1$.

(2)Commitment generation queries:

Attacker $\mathcal{A}_1$ adaptively initiates a commitment generation request to challenger $\mathcal{B}$ for access policy Γ . Challenger $\mathcal{B}$ generates the corresponding policy commitment value $Commit(p)$ and returns it to attacker $\mathcal{A}_1$.

Challenge: Attacker $\mathcal{A}_1$ submits a target policy Γ^* to challenger $\mathcal{B}$. Challenger $\mathcal{B}$ generates a policy commitment value $Commit^*(p)$ and returns it to attacker $\mathcal{A}_1$.

Phase2: Attacker $\mathcal{A}_1$ can continue to launch queries to challenger $\mathcal{B}$ for policy commitment values adaptively, but none of the access policies Γ obtained from queries satisfy the target policy Γ^* .

Verification: Challenger $\mathcal{B}$ verifies the final submitted commitment value $Commit(p)$ of attacker $\mathcal{A}_1$ against $Commit^*(p)$ of the challenge phase. If there exists a non-target attribute set commitment value that is equal to the target attribute set commitment value, then attacker $\mathcal{A}_1$ wins the challenge.

The probability that attacker $\mathcal{A}_1$ succeeds in the challenge is defined as:

$$Adv_{A_1}^{VC}(\lambda) = \left| Pr[Commit(p) = Commit^*(p)] - \frac{1}{2} \right|$$

If the probability $Adv_{A_1}^{VC}(\lambda)$ that attacker $\mathcal{A}_1$ wins the challenge is negligible in this polynomial time, then the scheme satisfies the attribute set commitment verification security.

Game2 Change of authority

Initialization: Challenger $\mathcal{B}$ runs the system initialization algorithm and generates the public parameters PP and the master key MK . Challenger $\mathcal{B}$ defines a collection of target attributes S^* .

Phase1: Attacker $\mathcal{A}_2$ can make adaptive queries with the following polynomial number:

(1)Key generation queries:

Attacker $\mathcal{A}_2$ adaptively initiates a query for attribute set S to challenger $\mathcal{B}$. Challenger $\mathcal{B}$ generates the corresponding attribute key AK_1^* and returns it to attacker $\mathcal{A}_2$.

(2)Re-encryption key query:

Attacker $\mathcal{A}_2$ adaptively initiates a query to challenger $\mathcal{B}$ for the set of attributes S . Challenger $\mathcal{B}$ generates the corresponding re-encryption key RK and returns it to attacker $\mathcal{A}_2$.

Challenge: Attacker $\mathcal{A}_2$ submits a collection of target attributes S^* to Challenger $\mathcal{B}$. Challenger $\mathcal{B}$ generates a re-encryption key RK^* and returns it to attacker $\mathcal{A}_2$.

Phase2: Attacker $\mathcal{A}_2$ can continue to launch adaptive queries for the re-encryption key to challenger $\mathcal{B}$. However, none of the attributes sets S obtained from the queries satisfies the target attribute set S^* .

Verification: Challenger $\mathcal{B}$ verifies that the re-encryption key RK finally submitted by attacker $\mathcal{A}_2$ is the same as RK^* in the challenge phase. If there exists a re-encryption key for the non-target attribute set that is equal to the re-encryption key for the target attribute set, then attacker $\mathcal{A}_2$ wins the challenge.

The probability that attacker $\mathcal{A}_2$ succeeds in the challenge is defined as:

$$Adv_{\mathcal{A}_2}^{VC}(\lambda) = \left| Pr[RK = RK^*] - \frac{1}{2} \right|$$

If the probability $Adv_{\mathcal{A}_2}^{VC}(\lambda)$ that attacker $\mathcal{A}_2$ wins the challenge is negligible in this polynomial time, then the scheme satisfies privilege change security.

Game3 Chosen plaintext attack (IND-CPA) security

Initialization: Challenger $\mathcal{B}$ runs the system initialization algorithm and generates the disclosure parameter PP and master key MK , and discloses PP .

Phase1: Attacker $\mathcal{A}_3$ can make adaptive queries with the following polynomial number:

Key Query Phase: Attacker $\mathcal{A}_3$ adaptively initiates a query to Challenger $\mathcal{B}$ for a collection of attributes S . The challenger $\mathcal{B}$ generates the corresponding attribute key AK'_1 and returns it to attacker $\mathcal{A}_3$. All attributes s_i do not satisfy the predefined challenge access policy Γ^* .

Challenge: Attacker $\mathcal{A}_3$ submits equal-length messages M_0 and M_1 , and challenger $\mathcal{B}$ randomly selects $\mu \in \{0, 1\}$ and runs a cryptographic algorithm that encrypts M_μ with a challenge access policy Γ^* to generate a CT^* , which is returned to attacker $\mathcal{A}_3$.

Phase2: Phase 1 repeats adaptively.

Guess: Attacker $\mathcal{A}_3$ outputs a guess μ' for μ . If $\mu' = \mu$, it means that attacker $\mathcal{A}_3$ wins the security game. The probability that attacker $\mathcal{A}_3$ succeeds in the challenge is defined as:

$$Adv_{\mathcal{A}_3}^{IND-CPA}(\lambda) = \left| Pr[\mu = \mu'] - \frac{1}{2} \right|$$

A scheme has indistinguishable security under the chosen-plaintext attack if the probability $Adv_{\mathcal{A}_3}^{IND-CPA}(\lambda)$ that attacker $\mathcal{A}_3$ wins the challenge is negligible in this polynomial time.

Design goals

To guarantee security and privacy, the proposed electronic health record sharing scheme is designed to fulfill the following design objectives.

(1)Correctness: Correctness serves as a fundamental pillar of data security, including CP-ABE ciphertexts, attribute-based proxy re-encryption, and so on. In addition, no PPT attacker can perform unauthorized access to the system or retrieve valid electronic health record data through such constructs;

(2)Verifiability: To safeguard the authenticity and usability of the EHR, the data user determines the authenticity of the data by the attribute promise value before decrypting the ciphertext;

(3)Flexible Commitment Changes: With the proxy re-encryption of attributes, the mechanism can efficiently handle dynamic changes in data user commitments, ensuring real-time and secure commitment adjustments;

(4)Fine-Grained Access Control: The DO has EHR commitments to sharing data as needed and defines access policies to specify authorized users of the EHRs.

Construction

The blockchain-based verifiable CP-ABE medical data privacy protection scheme can be divided into five phases: system construction, data encryption, data access and verification, data decryption, and dynamic change management of privileges. Figure 3 depicts the flow chart of the proposed scheme.

System construction

This process comprises two sub-phases: system initialization and key generation.

System initialization $Setup(1^\lambda, d) \rightarrow (PP, MK)$: The key generation center is used to execute this algorithm with the input security parameter λ and vector dimension d as follows:

1) Let $e: \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ denote a bilinear mapping, where $\mathbb{G}_1$ and $\mathbb{G}_2$ are cyclic multiplicative groups of prime order p , and let g be a generator of $\mathbb{G}_1$.

2) Define a collision-resistant hash function: $H: \{0, 1\}^* \rightarrow \mathbb{G}_1$.

3)Select $\alpha, \beta, Z_1, Z_2, Z_3, \dots, Z_d \in \mathbb{Z}_p^*$ randomly with master key $MK = (\alpha, \beta, Z_1, Z_2, Z_3, \dots, Z_d)$. Compute the public key parameters $h = g^\beta, e(g, g)^\alpha$, and $h_i = g^{Z_i}$ ($i = 1, 2, \dots, d$) for attribute-vector mapping. For attributes in the access policy, attribute-vector base mapping is first performed in two steps: 1) attribute position assignment, where each attribute is manually assigned a public position in the vector (e.g., “doctor” maps to position i and “dean” to position j in the access policy); 2) attribute value encoding, where user attributes are encoded as a d -dimensional binary vector $x = (x_1, x_2, \dots, x_d)$, with 1 indicating the presence of an attribute and 0 indicating absence. Define the d -dimensional vector commitment $VC = (g^{W_{vc}}, \prod_{i=1}^d h_i^{x_i})$, where $W_{vc} \in \mathbb{Z}_p^d$ is a random vector. For access tree nodes (logical gates), validation rules are as follows: for an AND gate with attributes A, B, C corresponding to vector bits m, n, k . For the joint encoding process, within the vector commitment rule, it is defined that the AND - relationship is satisfied if and only if all three bits equal 1. Based on this, the validation function is constructed as follows (Eq. 2):

$$e(g^{W_{vc}}, h_m, h_n, h_k) \stackrel{?}{=} e\left(\prod_{i'=1}^d h_{i'}^{x_{i'}}, g^{W_{vc} \cdot (e_m + e_n + e_k)}\right) \quad (2)$$

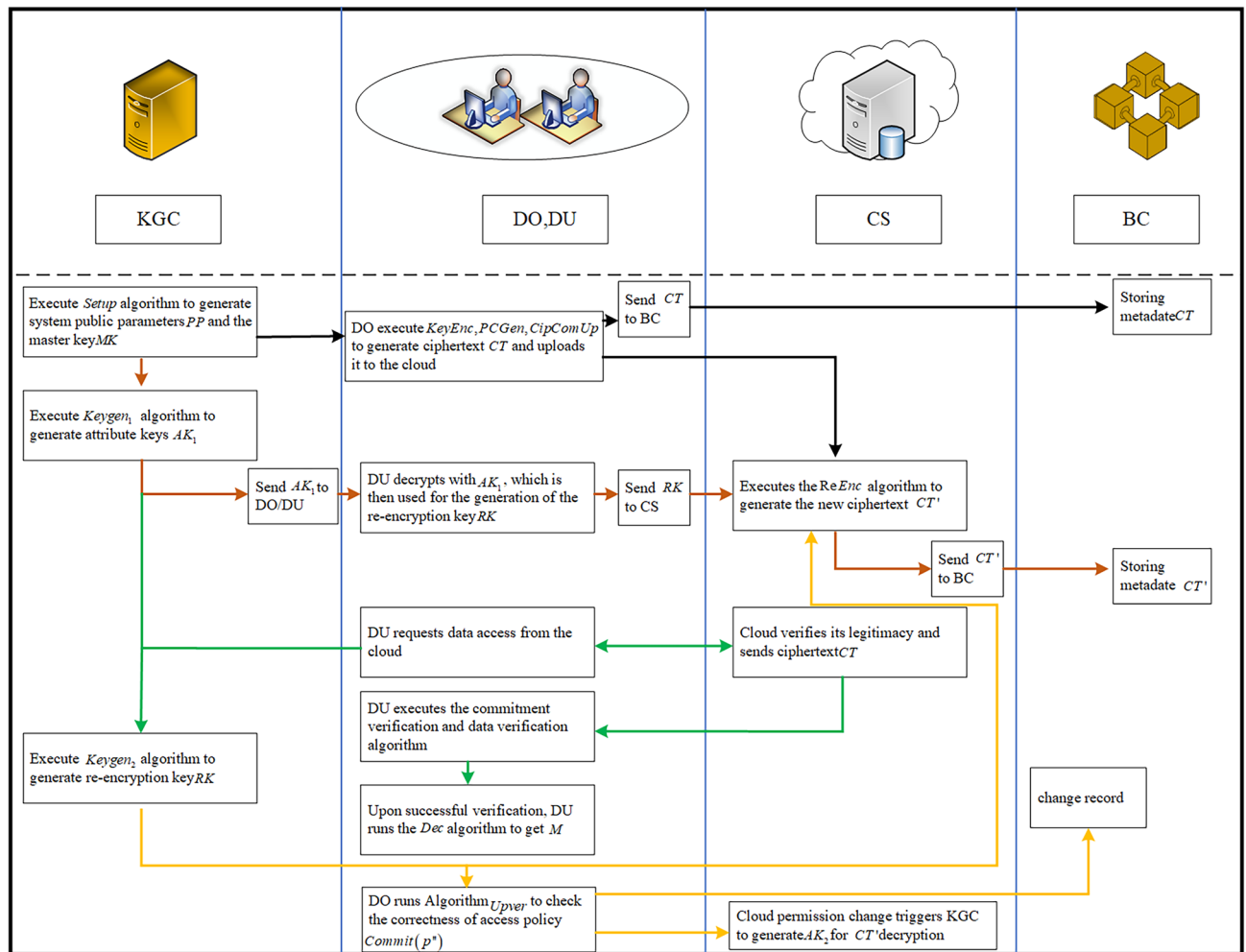


Figure 3. Schematic diagram of the program flowchart.

Here, e_m denotes the m -th bit unit vector. The equation holds, and verification succeeds if and only if $x_m = x_n = x_k = 1$; otherwise, if any bit is 0, the bit does not match, and verification fails. For the “OR” gate, suppose attributes D and E have an “OR” relationship, corresponding to the o -th and w -th bits of the vector; the validation function is constructed as follows (Eq. 3):

$$e(g^{\mathbf{w}_{vc}}, h_o, h_w) = e\left(\prod_{i'=1}^d h_{i'}^{x_{i'}}, g^{\mathbf{w}_{vc} \cdot (e_o + e_w)}\right) \quad (3)$$

If $x_o = 1$ or $x_w = 1$, then $h_o^0 = 1$ or $h_w^1 = 1$ must hold in $\prod_{i'=1}^d h_{i'}^{x_{i'}}$ for the equation to hold; if $x_o = x_w = 1$, then $h_o^0 = h_w^0 = 0$ in $\prod_{i'=1}^d h_{i'}^{x_{i'}}$ leads to a missing bit condition, causing validation to fail. Next, generate a random seed, compute $H_i = g^\alpha \cdot H(\text{seed} \oplus i)$ for subsequent policy commitment-related computations, and publish public parameters $PP = (\mathbb{G}_1, \mathbb{G}_2, g, h, e(g, g)^\alpha, \{h_i\}_{i=1}^d, \{H_i\}_{i=1}^d, \text{seed}, H(\cdot))$.

Key generation $Keygen_1(PP, MK, S) \rightarrow AK_1$: The key generation center comes to perform this algorithm to generate the attribute key AK_1 for the DU. First, the set of user attributes $\{S = s_1, s_2, \dots, s_d\}$ covers a set of specific attributes a user possesses, such as role attributes (e.g., doctor, dean), commitment attributes (e.g., prescription rights, decision-making rights), and so on. Subsequently, the traversal process is entered, i.e., starting from the first attribute s_1 of the attribute set S , the attributes are processed one by one until the last attribute s_d . In processing, each attribute s_i randomly selects $r \in \mathbb{Z}_p^*$ and $r_i \in \mathbb{Z}_p^*$ and computes $D = g^{\frac{\alpha}{\beta}} + g^{\frac{r}{\beta}} = g^{\frac{\alpha+r}{\beta}}$, $D_i = g^{\alpha+r} \cdot H(attr(s_i))^{r_i} \cdot (h_i)^{-1}$ and $D'_i = g^{r_i}$. The result is incorporated into constructing the key AK_1 and the final attribute key $AK_1 = \{D, \{D_i, D'_i\}_{s_i \in S}\}$ for each completed attribute.

Data encryption

This phase includes access tree construction, key encryption, policy commitment generation, and ciphertext upload.

Access tree construction $TreeGen(\cdot) \rightarrow \Gamma_1$: The data owner constructs the access tree Γ_1 before uploading the EHR according to the following rules. For every node y in the access tree Γ_1 , the operation process is opened from the root node $root$. First, a polynomial P_y of order d_y is selected for each node y . Here, the threshold value of the node y is set to t_y and $d_y = t_y - 1$ is specified. Starting from the root node $root$, pick a random element r in the set $\mathbb{Z}_p^*$ and make $P_{root}(0) = r$, then randomly pick coefficients d_{root} to determine the polynomial P_{root} . For the nodes y other than the root node, define them according to $P_y(0) = P_{parent(y)}(index(y))$, and also randomly pick d_y points for determining the polynomial P_y , and thus complete the construction of the access tree Γ_1 . In this access tree Γ_1 structure, the leaf nodes represent the attributes, and the internal nodes constitute the threshold conditions from their children nodes as well as the threshold combinations. Let the number of children of node y be n_y and the threshold value be t_y , which needs to satisfy $0 < t_y \leq n_y$. When $t_y = n_y$, this threshold gate is equivalent to an “AND” logical gate; when $t_y = 1$, this threshold gate is equivalent to an “OR” logical gate. We use $parent(y)$ to denote the parent node of node y , $attr(y)$ to denote the attributes of leaf node y , and $idx(y)$ to indicate the number associated with node. In this scheme, each leaf node corresponds to an attribute, e.g., “Doctor or Surgery,” each non-leaf node represents a logical operation, e.g., “AND or OR,” and the root node represents the logical result of the entire access policy. The shape of the access policy constructed by the access tree is shown in Fig. 4.

Key encryption $KeyEnc(M, PP) \rightarrow (C, C', C_y, C'_y)$: DO choose a random number $s \in \mathbb{Z}_p^*$ and compute the core part of the ciphertext, $C = M \cdot e(g, g)^{\alpha s}$, $C' = h^s$. M is the plaintext, C is the encrypted data, and C' is used to calculate bilinear pairs for decryption. For each node y in the access tree Γ_1 , compute $C_y = g^{P_y(0)}$, $C'_y = H(attr(y))^{P_y(0)}$. Where C_y is the ciphertext component corresponding to node y , and C'_y is the ciphertext component associated with the attribute corresponding to node y .

Policy commitment generation $PCGen(PP, \Gamma_1) \rightarrow Commit(p)$: the DO chooses the random number $y_e \in \mathbb{Z}_p^*$ and computes $Y_e = g^{y_e}$, which computes the commitment value $C_0 = \prod_{i=1}^d H_i^{t_i}$, where $i \in \{1, 2, \dots, d\}$, $H_i = g^{\alpha} \cdot t_i$, and $t_i = H(seed \oplus i)$. Next, the auxiliary information $H_a = H(\Gamma_1, C_0)^{Y_e}$ is calculated, and for each location i involved in the access policy Γ_1 , the corresponding proof π_i is generated as follows (Eq. 4):

$$\pi_i = \prod_{j \neq i} H_j^{t_j} \quad (4)$$

Finally, the policy commitment $Commit(p) = (C_0, H_a, \pi_i)$ is generated.

Cipher combination and upload $CipComUp(C, C', (C_y, C'_y)_{y \in \Gamma_1}, Commit(p)) \rightarrow CT$: DO uploads encrypted

$CT = (\Gamma_1, C, C', (C_y, C'_y)_{y \in \Gamma_1}, Commit(p))$ to the cloud for subsequent access and decryption.

Data access and validation

This phase can be divided into three parts: data access, commitment verification, and data validation.

Data access $DataAcc(PP, ID) \rightarrow CT$: DU sends a data access request to the cloud server, including the identifier of the required data (for example, filename or unique ID). The cloud server verifies the legitimacy of the DU. Then, it returns the

$CT = (\Gamma_1, C, C', (C_y, C'_y)_{y \in \Gamma_1}, Commit(p))$ along with the $Commit(p) = (C_0, H_a, \pi_i)$ required for verification, so there's no need to obtain the commitment value from the blockchain separately.

Commitment verification $ComVer(CT, PP) \rightarrow valid/Invalid$: The DU uses $Commit(p)$ in the ciphertext for verification. The key steps are outlined below: first, the DU regenerates the commitment value based on the access policy Γ_1 , the disclosure parameter PP , and the vector commitment algorithm in the

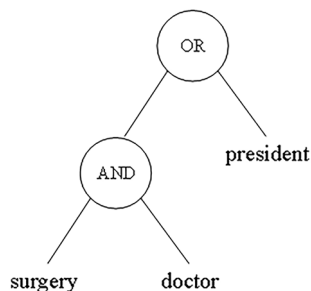


Figure 4. Γ_1 access tree.

downloaded ciphertext CT . The generation of the commitment value relies on the location boundedness of the vector commitment.

(1) The commitment value C'_0 is computed as follows (Eq. 5):

$$C'_0 = \prod_{i=1}^d [g^\alpha \cdot H(\text{seed} \oplus i)]^{H(\text{seed} \oplus i)} \quad (5)$$

(2) Verification needs to be satisfied (Eq. 6):

$$e(C'_0, g^{\alpha^{d+1-i}}) \stackrel{?}{=} e(\pi_i, Y_e^{\alpha^{d+1-i}}) \cdot e(H_i^{t_i}, g^{\alpha^{d+1-i}}) \quad (6)$$

Equation (4) ensures the consistency of π_i , the commitment value C'_0 , and Y_e . The relationship between π_i and $e(C'_0, g^{\alpha^{d+1-i}})$ is verified to confirm whether (C_0, H_a) matches the location involved in the access policy Γ_1 .

If all checks are consistent, the commitment verification passes; otherwise, the ciphertext may be tampered with or transmitted incorrectly, the verification fails, and the subsequent operation is terminated.

Data validation $\text{DataVer}(i, PP, v_i, \pi_i) \rightarrow \text{Valid/Invalid}$: The DU selects the location i that needs to be validated (e.g., corresponding to a certain attribute a_j), and based on the attribute-vector mapping relation h_i in the policy commitment, the DU invokes the algorithm $VC.Open$, which generates the value v_i and computes the proof π_i for the location i . Next, the DU uses the $VC.Ver$ algorithm, which is input with v_i , π_i , and the commitment value C_0 , to check the correctness of the (Eq. 7) below:

$$e(C_0, g) \stackrel{?}{=} e(\pi_i, g) \cdot e(H_i^{v_i}, g) \quad (7)$$

If the equation is valid, it means that v_i is the correct value of the location i . Otherwise, this signifies that the data has been tampered with.

Data decryption

Data decryption $\text{Dec}(PP, CT, AK_1) \rightarrow (M/\perp)$: DU uses its attribute key to decrypt the ciphertext CT to obtain the plaintext data M . Decryption relies on whether the user's attributes meet the access policy embedded as a tree in the ciphertext.

(1) Traverse each leaf node in the access tree Γ_1 and check if the set of user attributes S covers the attributes requested by the node; if the characteristics of any leaf node are not fulfilled, the decryption ends and returns $\perp$.

(2) Leaf node processing: each leaf node y corresponds to the attribute $\text{attr}(y)$, and if $\text{attr}(y) \in S$, the leaf node value F_y is computed using the attribute key components D_y and D'_y as shown in (Eq. 8):

$$\begin{aligned} F_y &= \frac{e(D_y, C_y)}{e(D'_y, C'_y)} \\ &= \frac{e(g^{\alpha+r} \cdot H(\text{attr}(y))^{r_y} \cdot g^{-Z_y}, C_y)}{e(g^{r_y} \cdot C'_y)} \\ &= e(g, g)^{\alpha P_y(0)} \end{aligned} \quad (8)$$

(3) Internal nodes: each internal node y , assuming that its children are $\{y_1, y_2, \dots, y_k\}$ and the values $F_{y_1}, F_{y_2}, \dots, F_{y_k}$ of the children have been computed recursively.

AND gate: all nodes have to satisfy $F_y = \prod_{i=1}^k F_{y_i}$

OR gate: any one of the child nodes y_i that satisfies it and takes the value $F_y = F_{y_i}$.

The threshold gate $((t_y, n_y)\text{threshold})$: at least t_y sub-nodes must be satisfied, and the secret value is recovered through Lagrangian interpolation as specified in (Eq. 9):

$$F_y = \prod_{i=1}^k F_{y_i}^{\Delta_i(0)} \quad (9)$$

Where $\Delta_i(0) = \prod_{j \in \{1, 2, \dots, k\}, j \neq i} \frac{0-j}{i-j}$ is the Lagrange interpolation coefficient.

(4) Calculate the root node value: recursively go to the root node $root$ to obtain $F_{root} = e(g, g)^{\alpha s}$, where s is a random number chosen during encryption to satisfy the polynomial $P_{root} = s$ at the root node of the access tree.

(5) Recover the plaintext: use the root node F_{root} to decrypt the core portion of the ciphertext C , obtaining the plaintext M . Calculate as follows (Eq. 10):

$$\begin{aligned} M &= \frac{C}{F_{root}} \\ &= \frac{M \cdot e(g, g)^{\alpha s}}{e(g, g)^{\alpha s}} \end{aligned} \quad (10)$$

Dynamic change management of privileges

This phase can be divided into re-encryption key generation, re-encryption execution, commitment update and verification, and commitment change notification.

Re-encryption key generation $Keygen_2(PP, AK_1, S', MK) \rightarrow RK$: The Key Generation Center performs this algorithm as follows:

(1) KGC selects the random number $\delta \in \mathbb{Z}_p^*$, generates the re-encryption component, and computes $RK_1 = g^\delta$ and $RK_2 = D^\delta = \left(g^{\frac{\alpha+r}{\beta}}\right)^\delta$.

(2) For each attribute $j \in S'$ in the user's new attribute set S' and revoked attribute subset $S_{rev} \in S$ if $j \in S \setminus S_{rev}$ (i.e., the subset of the original attribute set S after removing revoked subset S_{rev}), generate attribute-related components and compute $RK_j = (D_j)^\delta = \left(g^{\alpha+r} \cdot H(attr(j))^{r_j} \cdot (h_j)^{-1}\right)^\delta$, $RK'_j = (D'_j)^\delta = (g^{r_j})^\delta$; if $j \in S_{rev}$, output $RK_j = \perp$ and $RK'_j = \perp$.

(3) Output the re-encryption key: $RK = (RK_1, RK_2, (RK_j, RK'_j)_{j \in S'})$.

Re-encryption execution $ReEnc(PP, CT, RK, \Gamma_2) \rightarrow CT'$: The cloud server to perform this algorithm is as follows:

(1) Adjust the access tree Γ_1 to Γ_2 based on the new attribute set S' to ensure that S' satisfies Γ_2 . For example, assume that the changed Γ_2 is shown in Fig. 5.

That is, the access policy becomes: "The DU can decrypt only if the attribute set satisfies '(deputydirector $\wedge$ doctor) $\vee$ (president $\wedge$ (chiefdoctor $\vee$ (anesthesiologist $\vee$ nurse $\vee$ photographer)))'". The DO decrypts the access tree Γ_2 by defining the positions of 'Associate Director', 'Doctor', 'President', 'Chief Physician', 'Anesthesiologist', 'Nurse', and 'Camera Technician' as $j = 1, 2, 3, 4, 5, 6, 7$, respectively. Let the global branch selection tag $Tag \in \{0, 1\}$, where 0 represents Branch 1 and 1 represents Branch 2; the sub-OR branch selection $tag \in \{0, 1\}$, where 0 represents "chief doctor" and 1 represents the threshold branch. The policy commitment is generated as follows:

- Generate the core values of the strategy as specified in (Eq. 11):

$$C_0'' = \left(\prod_{j=2}^2 H_j^{t_j''} \right)^{Tag=0} \cdot \left(H_3^{t_3} \cdot \left(H_4^{t_4 \cdot (1-tag)} \cdot \prod_{j=5}^7 (g^{\alpha \cdot \Delta_j(0)} \cdot H_j)^{t_j \cdot \sigma_j \cdot tag} \right) \right)^{Tag=1} \quad (11)$$

$$= \begin{cases} g^{\alpha(t_1+t_2)} H_1^{t_1} H_2^{t_2}, & Tag = 0 \\ g^{\alpha(t_3+t_4(1-tag)+tag \sum_{j=5}^7 t_j \cdot \sigma_j \cdot \Delta_j(0))} H_3^{t_3} H_4^{t_4(1-tag)} \prod_{j=5}^7 H_j^{t_j \cdot \sigma_j \cdot tag \cdot \Delta_j(0)}, & Tag = 1 \end{cases}$$

where t_j is the threshold weight and σ_j denotes the secret share of the attribute node;

- Generate the auxiliary verification quantity $H_a'' = H(\Gamma_2, C_0'')$ and the proof value π_i'' . Finally, set the policy commitment as $Commit(p'') = (C_0'', H_a'', \pi_i'')$; (2) Compute the main parameters of the new ciphertext as follows (Eq. 12):

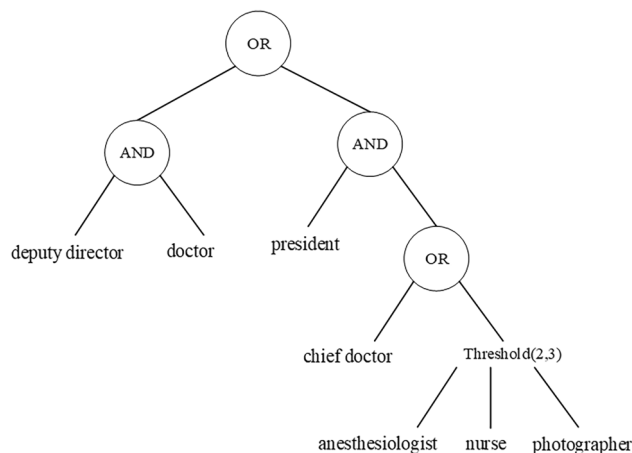


Figure 5. Γ_2 access tree.

$$\begin{aligned}
C_n &= C' \cdot e(RK_1, RK_2) \cdot SAF(\Gamma_2) \\
&= h^s \cdot e\left(g^\delta, g^{\left(\frac{\alpha+r}{\beta}\right)\delta}\right) \cdot e(g, g)^{\frac{\alpha \cdot Tag \cdot tag}{\beta}} \\
&= h^s \cdot e(g, g)^{\left(\frac{\alpha+r}{\beta}\right)\delta^2} \cdot e(g, g)^{\frac{\alpha \cdot Tag \cdot tag}{\beta}} \\
&= h^s \cdot e(g, g)^{\frac{\alpha(\delta^2 + Tag \cdot tag \cdot \sigma) + r\delta^2}{\beta}}
\end{aligned} \tag{12}$$

where SAF is the secret recovery value of the policy tuning factor, and the threshold branch $\sigma = \sum_{j=1}^{t_y} \sigma_j \cdot \Delta_j(0)$.

(3) For each attribute node y (corresponding to attribute $y \notin S_{rev}$) in the access tree Γ_2 , generate the re-encrypted attribute components as follows: $C_y'' = C_y \cdot e(RK_j, C_y' \cdot e(g, g)^{\sigma_j \cdot Tag \cdot tag})$, $C_y''' = C_y' \cdot e(RK_j', C_y \cdot (1 - tag)^{\delta(j=4)})$ where

$C_y = g^{p_y(0)}$, $C_y' = H(attr(y))^{p_y(0)}$. If attribute $j \in S_{rev}$ (i.e., the corresponding access tree Γ_2 revokes the attribute from the original access tree Γ_1), the generated re-encrypted attribute components are invalid, i.e., $C_y'' = \emptyset$, $C_y''' = \emptyset$.

(4) Generate the new ciphertext $CT = (\Gamma_2, C, C_n, (C_y', C_y'')_{y \in \Gamma_2}, Commit(p''))$.

Commitment update and verification $UpVer(PP, CT', \Gamma_2) \rightarrow 1/0$: DO inputs the public parameter PP , the new ciphertext CT' , and the new policy Γ_2 . The computation is as follows (Eq. 13):

$$\begin{aligned}
Commit(p^*) &= Com(cp, CT') \\
&= (C_0^*, H_a^*, \pi_i^*) \\
&= \left(\prod_{i=1}^d H_1^{t_i^*} \cdot \prod_{i=1}^d H_2^{t_i^*}, H(\Gamma_2, C_0^*)^{Y_e}, \pi_i^* \right)
\end{aligned} \tag{13}$$

Commit $Commit(p^*)$ and other relevant information to the blockchain, then DO downloads CT' , obtains $Commit(p'')$ from the blockchain, and executes the commitment verification algorithm: $C_0^* = \prod_{i=1}^d H_1^{t_i^*} \cdot \prod_{i=1}^d H_2^{t_i^*}$, $H_a^* = H(\Gamma_2, C_0^*)^{Y_e}$, $\pi_i^* = \pi_i^*$. If all three are equal, input 1 to show that the ciphertext is complete; otherwise, input 0 to show that the ciphertext has been tampered with.

Commitment change notification $PerChaNot(S', PP) \rightarrow AK_2$: When the blockchain finishes storing the commitment change records, the cloud server actively retrieves the change information on the blockchain. After detecting the corresponding record, it automatically sends a notification to the target user. Based on the updated attribute set S' , the user applies for a new attribute key AK_2 from the KGC. The user decrypts the CT' using AK_2 and confirms the integrity of the CT' by using the $UpVer$ algorithm.

Security analysis

In this subsection, we formally analyze the security of our blockchain-based verifiable CP-ABE scheme for healthcare data, focusing on three key aspects: correctness, attack resistance and cryptographic assumptions.

Proof of correctness

The correctness of the proposed scheme can be proved by assuming that all entities in the system can correctly generate the security parameters and successfully compute the security key.

Proof In the data access and validation phase, the correctness of Eq. (6) is verified to confirm that (C_0, H_a) matches the location involved in the access policy Γ_1 .

Left calculation: (Eq. 14)

$$\begin{aligned}
&e\left(C_0', g^{\alpha^{d+1-i}}\right) \\
&= e\left(\prod_{j \neq i}^d H_j^{t_j} \cdot H_i^{t_i}, g^{\alpha^{d+1-i}}\right) \\
&= e\left(\pi_i \cdot H_i^{t_i}, g^{\alpha^{d+1-i}}\right) \\
&= e\left(\pi_i, Y_e^{\alpha^{d+1-i}}\right) \cdot e\left(H_i^{t_i}, g^{\alpha^{d+1-i}}\right)
\end{aligned} \tag{14}$$

Right calculation: (Eq. 15)

$$e\left(\pi_i, Y_e^{\alpha^{d+1-i}}\right) \cdot e\left(H_i^{t_i}, g^{\alpha^{d+1-i}}\right) = e\left(\pi_i, Y_e^{\alpha^{d+1-i}}\right) \cdot e\left(H_i^{t_i}, g^{\alpha^{d+1-i}}\right) \quad (15)$$

In the scheme design, Eq. (6) holds for the left and right sides, and the commitment verification passes, indicating that the ciphertext has not been tampered with and matches the access policy. $\square$

Proof In the data access and validation phase, it is confirmed that the data has not been tampered with by verifying the correctness of Eq. (7). If the equation holds, it signifies that the data remains untampered and retains its integrity and authenticity.

Left calculation: (Eq. 16)

$$\begin{aligned} & e(C_0, g) \\ &= e\left(H_i^{t_i} \cdot \prod_{j \neq i} H_j^{t_j}, g\right) \\ &= e\left(H_i^{t_i} \cdot \pi_i, g\right) \\ &= e(\pi_i, g) \cdot e\left(H_i^{t_i}, g\right) \end{aligned} \quad (16)$$

Right calculation: (Eq. 17)

$$\begin{aligned} & e(\pi_i, g) \cdot e\left(H_i^{t_i}, g\right) \\ &= e\left(\prod_{j \neq i} H_j^{t_j}, g\right) \cdot e\left(H_i^{t_i}, g\right) \\ &= \prod_{j \neq i} e\left(H_j^{t_j}, g\right) \cdot e\left(H_i^{t_i}, g\right) \\ &= e\left(\prod_{j \neq i} H_j^{t_j} \cdot H_i^{t_i}, g\right) \\ &= e(C_0, g) \end{aligned} \quad (17)$$

In the design of the scheme, t_i is the correct value when the strategy is committed to C_0 , representing the true attribute value associated with location i . At the same time, v_i is the actual value used for comparison in the verification phase. The above equation holds if and only if $v_i = t_i, H_i^{v_i} = H_i^{t_i}$. $\square$

Proof In the data decryption process for leaf nodes, we determine whether the value of the leaf node can be successfully decrypted by verifying the correctness of Eq. (8). If the equation holds, it shows that the value of the leaf node can be decrypted correctly, as demonstrated in the verification process of (Eq. 18).

$$\begin{aligned} F_y &= \frac{e(D_y, C_y)}{e(D'_y, C'_y)} \\ &= \frac{e\left(g^{\alpha+r} \cdot H(attr(y))^{r_y} \cdot g^{-Z_y}, C_y\right)}{e(g^{r_y}, C'_y)} \\ &= \frac{e\left(g^{\alpha+r-Z_y} \cdot H(attr(y))^{r_y}, C_y\right)}{e(g^{r_y}, C'_y)} \\ &= \frac{e\left(g^{\alpha+r-Z_y} \cdot H(attr(y))^{r_y}, g^{P_y(0)}\right)}{e(g^{r_y}, H(attr(y))^{P_y(0)})} \\ &= \frac{e(g, g)^{(\alpha+r-Z_y)P_y(0)} \cdot e\left(H(attr(y))^{r_y}, g^{P_y(0)}\right)}{e(g^{r_y}, H(attr(y))^{P_y(0)})} \\ &= \frac{e(g, g)^{\alpha P_y(0)} \cdot e\left(H(attr(y))^{r_y}, g^{P_y(0)}\right)}{e(g^{r_y}, H(attr(y))^{P_y(0)})} \\ &= e(g, g)^{\alpha P_y(0)} \cdot \frac{e\left(H(attr(y))^{r_y}, g^{P_y(0)}\right)}{e(g^{r_y}, H(attr(y))^{P_y(0)})} \\ &= e(g, g)^{\alpha P_y(0)} \end{aligned} \quad (18)$$

As a result, Eq. (8) holds and the values of the leaf nodes can be derived successfully. That is, the proposed scheme's decryption of leaf nodes is correct.

In summary, by verifying the correctness of Eqs. (6), (7) and (8), we can confirm the effectiveness of the blockchain-based verifiable CP-ABE healthcare data privacy protection scheme in the data access verification and decryption process. This demonstrates that the proposed scheme guarantees data integrity and authenticity while enabling secure, reliable decryption operations, thereby validating its correctness and effectiveness. $\square$

Proof of security

Game1 Strategy commitment verification security

In this article, we design a security game between attacker $\mathcal{A}_1$ and challenger $\mathcal{B}$ based on the difficulty of the DLP problem. By simulating the attack and defense interactions, we verify that the algorithm guarantees the security of the policy commitment verification with a “negligible probability of success for the attacker.”

Initialization: Challenger $\mathcal{B}$ runs the system initialization algorithm, defines the multiplicative group $\mathbb{G}_1 = \mathbb{Z}_p^*$ and selects the generating element $g \in \mathbb{G}_1$. The public parameter $PP = (\mathbb{G}_1, \mathbb{G}_2, g, h, e(g, g)^\alpha, \{h_i\}_{i=1}^d, \{H_i\}_{i=1}^d, seed, H(\cdot))$ and the master key $MK = (\alpha, \beta, Z_i)$ are generated and sent to $\mathcal{A}_1$. Challenger $\mathcal{B}$ defines the target access policy, Γ^* .

Phase1: Attacker $\mathcal{A}_1$ can make adaptive queries with the following polynomial number:

(1)Key generation queries:

Attacker $\mathcal{A}_1$ submits an access policy Γ where $\Gamma \neq \Gamma^*$. The challenger $\mathcal{B}$ generates the corresponding attribute key

$$AK_1' = \left\{ D' = g^{\frac{\alpha+r'}{\beta}}, \left\{ D_i' = g^{\alpha+r} \cdot H(\text{attr}(s_i))^{r_i'} \cdot (h_i)^{-1}, D_i'' = g^{r_i'} \right\}_{s_i \in S} \right\}$$

using the secret values α and β from the master key MK , random numbers $r' \in \mathbb{Z}_p^*$, $r_i' \in \mathbb{Z}_p^*$, and $y_e' \in \mathbb{Z}_p^*$, along with the collection of user attributes $S = \{s_1, s_2, \dots, s_d\}$. The attacker $\mathcal{A}_1$ can initiate such queries multiple times. However, due to the one - way property of the DLP, $\mathcal{A}_1$ cannot deduce the key secret value α .

(2)Commitment generation queries:

Attacker $\mathcal{A}_1$ submits an access policy Γ . Challenger $\mathcal{B}$ executes the policy commitment generation algorithm $PCGen$ and computes the commitment value $Commit(p) = (C_0 = \prod_{i=1}^d H_i^{t_i}, H_a = H(\Gamma_1, C_0)^{Y_e}, \pi_i)$ according to the rule. Challenger $\mathcal{B}$ generates the corresponding policy commitment value $Commit(p)$ and returns it to attacker $\mathcal{A}_1$.

Challenge: Attacker $\mathcal{A}_1$ launches a challenge against the target access policy Γ^* defined by challenger $\mathcal{B}$. Challenger $\mathcal{B}$ generates the corresponding policy commitment value $Commit^*(p)$ and returns it to attacker $\mathcal{A}_1$.

Phase2: Attacker $\mathcal{A}_1$ continues to initiate policy inquiries. The challenger $\mathcal{B}$ ensures that the returned policy commitments do not involve Γ^* , restricting attacker $\mathcal{A}_1$ from obtaining the core commitment information of the target policy.

Verification: Define two states of $Commit(p)$:

$Commit(p) = 0$: indicates a random state where the attacker submits a non - target policy commitment.

$Commit(p) = 1$: indicates the attacker's attempt to construct a commitment value associated with the target policy commitment.

Attacker $\mathcal{A}_1$ outputs a guess. Assuming that the probability that attacker $\mathcal{A}_1$ generates algorithmic keyword information from a strategic commitment is $Adv_{A_1}^{VC}(\lambda)$, the probability that challenger $\mathcal{B}$ solves the DLP problem in the above challenge is (Eq. 19):

$$\begin{aligned} Adv_{A_1}^{DLP}(\lambda) &= \left| \frac{1}{2} Pr[Commit(p) = Commit^*(p) | Commit(p) = 0] + \frac{1}{2} Pr[Commit(p) = Commit^*(p) | Commit(p) = 1] - \frac{1}{2} \right| \\ &= \left| \left[\frac{1}{2} \times \left(\frac{1}{2} + Adv_{A_1}^{VC} \right) + \frac{1}{2} \times \frac{1}{2} \right] - \frac{1}{2} \right| \\ &= \frac{1}{2} Adv_{A_1}^{VC}(\lambda) \end{aligned} \quad (19)$$

Assuming that $Adv_{A_1}^{VC}(\lambda)$ is non-negligible, $Adv_{A_1}^{DLP}(\lambda)$ is non-negligible, which contradicts the Hardship Assumption in the DLP problem. Shows that strategic commitment verification is safe.

Game2 Commitment change security

In this article, we guarantee the security of the change in privileges by building an algorithm to solve the DBDH problem. This is demonstrated by simulating a security game between attacker $\mathcal{A}_2$ and challenger $\mathcal{B}$, who are trying to break the privilege changes.

Initialization: Challenger $\mathcal{B}$ runs the system initialization algorithm, defines the multiplicative group $\mathbb{G}_1 = \mathbb{Z}_p^*$ and selects the generating element $g \in \mathbb{G}_1$. The public parameter $PP = (\mathbb{G}_1, \mathbb{G}_2, g, h, e(g, g)^\alpha, \{h_i\}_{i=1}^d, \{H_i\}_{i=1}^d, seed, H(\cdot))$ and the master key $MK = (\alpha, \beta, Z_i)$ are generated and sent to $\mathcal{A}_2$ defines the attribute set, S^* .

Phase1: Attacker $\mathcal{A}_2$ can make adaptive queries with the following polynomial number:

(1)Key generation queries:

Attacker $\mathcal{A}_2$ submits an attribute set S , where $S \neq S^*$. The challenger $\mathcal{B}$ generates the corresponding attribute key

$$AK'_1 = \left\{ D' = g^{\frac{\alpha+r'}{\beta}}, \left\{ D'_i = g^{\alpha+r} \cdot H(\text{attr}(s_i))^{r'_i} \cdot (h_i)^{-1}, D''_i = g^{r'_i} \right\}_{s_i \in S} \right\}$$

using the secret values α and β from the master key MK , the random numbers $r' \in \mathbb{Z}_p^*$, $r'_i \in \mathbb{Z}_p^*$, and $y'_e \in \mathbb{Z}_p^*$, along with the set of user attributes $S = \{s_1, s_2, \dots, s_d\}$. The attacker $\mathcal{A}_2$ can initiate queries of this type multiple times. However, each time the queried attribute set S does not satisfy the target attribute set S^* .

(2) Re-encryption key query:

Attacker $\mathcal{A}_2$ submits the collection of attributes S . Challenger $\mathcal{B}$ performs the re-encryption key generation algorithm $KeyGen_2$ and generates the re-encryption key $RK = (RK_1, RK_2, (RK_j, RK'_j)_{j \in S_2})$ according to the rules, where $RK_1 = g^\delta$, $RK_2 = D^\delta = (g^{\frac{\alpha+r}{\beta}})^\delta$, $RK_j = (D_j)^\delta = (g^{\alpha+r} \cdot H(\text{attr}(j))^{r_j} \cdot (h_j)^{-1})^\delta$, $RK'_j = (D'_j)^\delta = (g^{r_j})^\delta$. Challenger $\mathcal{B}$ generates the corresponding re-encryption key RK and returns it to attacker $\mathcal{A}_2$.

Challenge: Attacker $\mathcal{A}_2$ submits a collection of target attributes S^* to challenger $\mathcal{B}$. Challenger $\mathcal{B}$ generates a re-encryption key RK^* and returns it to attacker $\mathcal{A}_2$.

Phase2: Attacker $\mathcal{A}_2$ continues to run query sets for attributes. Meanwhile, challenger $\mathcal{B}$ ensures that the returned attribute sets do not contain elements from set S^* . This measure effectively restricts attacker $\mathcal{A}_2$ from obtaining the core information of the re-encryption key RK^* .

Verification: Define two states of RK :

$RK = 0$: indicates that the re-encryption key is a random state independent of the target attribute set;

$RK = 1$: indicates that the attacker $\mathcal{A}_2$ tries to construct a key related to RK^* .

Challenger $\mathcal{B}$ verifies that the re-encryption key RK finally submitted by attacker $\mathcal{A}_2$ is the same as RK^* in the challenge phase. If non-target and target re-encryption keys are the same, $\mathcal{A}_2$ wins the challenge.

The attacker outputs a guess. Assuming that the probability of attacker $\mathcal{A}_2$ keywording information from the generating algorithm of the re-encryption key is $Adv_{A_2}^{PC}(\lambda)$, the probability that challenger $\mathcal{B}$ solves the DBDH problem in the above challenge is (Eq. 20):

$$\begin{aligned} Adv_{A_2}^{DBDH}(\lambda) &= \left| \frac{1}{2} Pr[RK = RK^* | RK = 0] + \frac{1}{2} Pr[RK = RK^* | RK = 1] - \frac{1}{2} \right| \\ &= \left| \left[\frac{1}{2} \times \left(\frac{1}{2} Adv_{A_2}^{PC}(\lambda) \right) + \frac{1}{2} \times \frac{1}{2} \right] - \frac{1}{2} \right| \\ &= \frac{1}{2} Adv_{A_2}^{PC}(\lambda) \end{aligned} \quad (20)$$

Assuming that $Adv_{A_2}^{PC}(\lambda)$ is non-negligible, then $Adv_{A_2}^{DBDH}(\lambda)$ is non-negligible, contradicting the difficulty assumption in the DBDH problem. Show that the program satisfies commitment change security.

Game3 Chosen plaintext attack (IND-CPA) security

Within this article, we formally prove the scheme's security against chosen-plaintext attack scenarios by constructing a security game model (which simulates attacker $\mathcal{A}_3$ attempting to break the plaintext and interacting with challenger $\mathcal{B}$) and designing the corresponding algorithms based on this model.

Initialization: Challenger $\mathcal{B}$ runs the system initialization algorithm, generates the disclosure parameter

$PP = (\mathbb{G}_1, \mathbb{G}_2, g, h, e(g, g)^\alpha, \{h_i\}_{i=1}^d, \{H_i\}_{i=1}^d, seed, H(\cdot))$ and master key $MK = (\alpha, \beta, Z_i)$, and discloses PP .

Phase1: Attacker $\mathcal{A}_3$ can make adaptive queries under the following polynomial number:

Key Query Phase: The attacker $\mathcal{A}_3$ submits an attribute set S , where $S \neq S^*$. The challenger $\mathcal{B}$ generates the corresponding attribute key

$$AK'_1 = \left\{ D' = g^{\frac{\alpha+r'}{\beta}}, \left\{ D'_i = g^{\alpha+r} \cdot H(\text{attr}(s_i))^{r'_i} \cdot (h_i)^{-1}, D''_i = g^{r'_i} \right\}_{s_i \in S} \right\}$$

using the secret values α and β from the master key MK , the random numbers $r' \in \mathbb{Z}_p^*$, $r'_i \in \mathbb{Z}_p^*$, and $y'_e \in \mathbb{Z}_p^*$, along with the collection of user attributes $S = \{s_1, s_2, \dots, s_d\}$.

Challenge: Attacker $\mathcal{A}_3$ submits equal-length messages M_0, M_1 and challenger $\mathcal{B}$ randomly selects $\mu \in \{0, 1\}$ and runs algorithms $KeyEnc, PCGen, CipComUp$. Then, it encrypts the M_μ with a challenge access policy Γ^* to generate

$CT^* = (\Gamma^*, C, C', (C_y, C'_y)_{y \in \Gamma_1}, Commit(p))$, which are returned to attacker $\mathcal{A}_3$.

Phase2: Phase 1 repeats adaptively.

Guess: Attacker $\mathcal{A}_3$ outputs a guess μ' for μ . If $\mu' = \mu$, challenger $\mathcal{B}$ succeeded in his challenge, input 1; otherwise, input 0.

Analysis: Attacker $\mathcal{A}_3$ guesses the hypothesis. If $v = 0$, the advantage of attacker $\mathcal{A}_3$ is $\varepsilon = Pr[\mu' = \mu] - \frac{1}{2}$, and the probability that challenger $\mathcal{B}$ wins is $Pr[\mu' = \mu | v = 1] = Pr[\mu' = \mu] = \varepsilon + \frac{1}{2}$. Otherwise, the probability that challenger $\mathcal{B}$ wins is $Pr[\mu' = \mu | v = 0] = Pr[\mu' \neq \mu] = \frac{1}{2}$. Then, the possibility that challenger $\mathcal{B}$ succeeds in the challenge is defined as (Eq. 21):

$$\begin{aligned}
& Adv_{A_3}^{IND-CPA}(\lambda) \\
&= \left| Pr[\mu' \neq \mu] - \frac{1}{2} \right| \\
&= \left| \frac{1}{2} (Pr[\mu' = \mu | v = 1] + Pr[\mu' = \mu | v = 0]) - \frac{1}{2} \right| \\
&= \left| \frac{1}{2} \left((1 - \varepsilon) + \frac{1}{2} \right) - \frac{1}{2} \right| \\
&= \frac{\varepsilon}{2}
\end{aligned} \tag{21}$$

Showing that the probability $Adv_{A_3}^{IND-CPA}(\lambda)$ that attacker $\mathcal{A}_3$ wins the challenge is negligible, the scheme has indistinguishable security under the choice of plaintext attack.

Security attributes

A. Replay attack

In this scheme, the auxiliary information H_a'' relies on the blockchain to record operation logs to ensure the uniqueness and non-tampering of policy-related operations. Each time a data operation (e.g., update) involves an access policy change, a new policy commitment value $Commit(p'')$ is generated and embedded in a new ciphertext CT'' . The policy commitment value in the old ciphertext is invalidated with the policy update. Suppose the attacker replays the ciphertext carrying the old policy commitment. In that case, the authentication session will fail because the value of the commitment to the policy $Commit(p'')$ within the ciphertext does not match the current valid policy. Therefore, this scheme can effectively resist replay attacks and ensure the safety and completeness of information.

B. Data tampering attack

In this scheme, the commitment value $C_0 = \prod_{i=1}^d H_i^{t_i}$ binds each data item v_j to a specific location j , ensuring that any modification of the data results in a new commitment value $C'_0 \neq C_0$. The auxiliary information $H_a = H(\Gamma_1, C_0)^{Y_e}$ associates the commitment value with the ciphertext CT , ensuring that any tampering with the ciphertext destroys the consistency of the hash. Furthermore, the accuracy of location-specific data can be confirmed by verifying the formula $e(C_0, g) \stackrel{?}{=} e(\pi_i, g) \cdot e(H_i^{v_i}, g)$. Suppose an attacker tries to tamper with the ciphertext or promised values stored on the cloud server. In that case, these changes will not be able to pass the above validation process, so this scheme can effectively defend against data tampering attacks.

C. Collusion attack

Theorem 1 Assume that the Discrete Logarithm Problem (DLP) assumption in group $\mathbb{G}_1$ holds and the attribute key generation satisfies the uniqueness of random numbers (each user private key contains an independent random factor). Then, in this scheme, the probability that multiple malicious users conspire to construct a valid decryption key and break the access control is negligible.

Proof Suppose there exists a polynomial-time adversary $\mathcal{A}$ that attempts to decrypt the data by colluding (controlling m users) to construct a valid private key, and $\mathcal{A}$ has a non-negligible advantage in experiments on the original database. For given public parameters $PP = (\mathbb{G}_1, \mathbb{G}_2, g, h, e(g, g)^\alpha, \{h_i\}_{i=1}^d, \{H_i\}_{i=1}^d, seed, H(\cdot))$, the master key related parameters $\alpha, \beta \in \mathbb{Z}_p^*$ are used as inputs. Under the DLP assumption, for any adversary $\mathcal{A}$, there exists a negligible function $negl(\lambda)$ such that the advantage is negligible. The construction of a user's attribute key $AK_1 = (D = g^{\frac{\alpha+r}{\beta}}, D_i = g^{\alpha+r} \cdot H(attr(s_i))^{r_i} \cdot (h_i)^{-1}, D'_i = g^{r_i})$ is known. Assume that adversary $\mathcal{A}$ tries to collude with the key components of t_{coll} users, constructs a new attribute key AK_{mal} , and challenges the integrity of the attribute key. The challenging process enables the adversary to obtain the random numbers r and r_i . Under the Discrete Logarithm Problem (DLP) assumption, the probability that AK_{mal} constructed by adversary $\mathcal{A}$ can be verified by the access tree Γ_1 privileges and can decrypt the ciphertext CT is given by (Eq. 22):

$$\Pr \left[\begin{array}{l} \text{Setup}(1^\lambda) \leftarrow (MK, PP) \leftarrow \mathcal{A}(PK) : \\ \text{KeyGen}_1(MK, PP, D, D_i, D'_i, AK_1) = 1 \wedge \\ \text{KeyGen}_1(MK, PP, D_{mal}, D_{mal,i}, D'_{mal,i}, AK_{mal}) = 1 \wedge AK_1 \neq AK_{mal} \end{array} \right] = \text{negl}(\lambda) \tag{22}$$

□

Performance analysis

This section analyzes the performance of the proposed scheme from three dimensions: functional comparison, theoretical analysis, and experimental evaluation. To comprehensively assess the scheme's effectiveness, it compares and analyzes representative schemes for similar data-sharing scenarios, including reference²² and reference²³ which focus on data validation mechanisms, and reference³⁵ which is oriented to privilege dynamic-change scenarios.

Functional comparison

From the functional level analysis, the scheme in this article relies on the vector commitment technique to extend the access control of EHR data. With this technology, data owners can enforce fine-grained access control on private data and generate unique, bound commitment values for access policies. This technical feature allows the system to monitor the integrity of the access policy in real-time, effectively resist the risk of tampering with it, ensure that any illegal changes to the access policy can be detected promptly, accurately verify the authenticity of the access policy, and safeguard the security of the data. In the security interaction link, the scheme provides key and ciphertext re-encryption services for different system roles through the AB-PRE mechanism. This effectively solves the problem of privilege adaptation when sharing cross-departmental data and makes sure the security of the data flow between different departments. Table 2 gives a functional comparison between our scheme and reference²², reference²³ and reference³⁵.

Theoretical analysis

Computational overhead analysis

we have theoretically analyzed the computational overhead of this scheme versus the references^{22,23}, and³⁵ and compiled the comparison results, as shown in Table 3.

In this scheme, the computational overhead mainly depends on the stages of the *PCGen* algorithm, the proof verification (*ComVer* and *DataVer*) algorithm stages, and the permission change management (*Keygen*₂, *ReEnc*, and *UpVer*) algorithm stages. The *PCGen* algorithm is executed by the DO, and the computational overhead is mainly used to generate the relevant parameters of the policy commitment *Commit*(*p*). In the proof generation stage, since the generated proof π_i is also included in the policy commitment *Commit*(*p*), the computational overhead of the *PCGen* algorithm stage is higher than that of the compared reference²³. In the proof verification stage, this scheme involves commitment verification and data verification. The computational overhead is $6P + (d + 2) M_{G_1} + dE_{G_1} + dH_{Z_p}$. Since reference²³ only involves one of them, no comparison is made; in addition, although the computational overhead of this scheme in this stage is not the lowest, it is within a reasonable range and is acceptable. In the permission change management stage, the computational overhead mainly depends on the *ReEnc* and *Keygen*₂ stages for generating the attribute-related components of the re-encryption key *RK*. The *ReEnc* algorithm stage handles the dynamic changes of attribute addition, modification, and revocation. The *UpVer* algorithm stage performs the comparison information of policy commitments. The total computational overhead of this stage is $(2d - 1) M_{G_1} + (2d + 1) E_{G_1} + H_{Z_p}$, which is higher than that of other stages and related to the data dimension, but still slightly lower than that of the compared reference²³.

Communications overhead analysis

Communication overhead incurred during EHR data sharing mainly consists of two parts: data access and validation, and data decryption. We compared the communication overhead of the proposed scheme with those of reference²², reference²³, and reference³⁵, respectively. The comparison results are presented in Table 4.

In phase *CipComUp*, besides the communication overhead from uploading the ciphertext *CT*, additional overhead arisen by uploading the ciphertext *CT*, the additional communication overhead incurred by the scheme is mainly used for uploading the metadata to the blockchain and the file auxiliary information to the CS, where the metadata contains the core part of the ciphertext and the ciphertext components as well as the policy commitment value. The core part of the ciphertext is only related to the chosen group G_1 . The policy commitment is associated with the data dimension and hash function, and the overhead of the ciphertext uploading phase is $(d + 2) |G_1| + 2 |Z_P^*| + |CT| + (d + 1) |H|$. In the data-sharing phase, the overhead of the verification and decryption parts is mainly concentrated on group G_1 and the data dimensions, which have an overhead of $(d + 10) |G_1| + 3 |Z_P^*| + (2d + 2) |H|$. In the update phase, the communication overhead is included in the *ReEnc* and *Upver* algorithm phases, which have an overhead of $(2d + 3) |G_1| + 4 |Z_P^*| + 5 |H|$. As shown in Table 4, the comparison reveals that this article's communication overhead is lower than that of the reference²², the reference²³ and the reference³⁵ due to the direct step of policy commitment generation, the specific way of proof generation, the compact logic of privilege updating and validation computation, the concise blockchain interactions, and the reduction of data interactions and computations.

Experimental evaluation

In this section, we experimentally evaluate the performance of the proposed scheme, as well as the references^{22,23} and³⁵, with the aim of verifying the intuition in the theoretical analysis. For the experimental setup, the proposed program is run on a laptop computer configured with the *IntelCorei7 - 5500UCPU@2.40GHz*, *8GBRAM* experimental setup, using a virtual machine environment under the *Ubuntu18.04* operating system. The

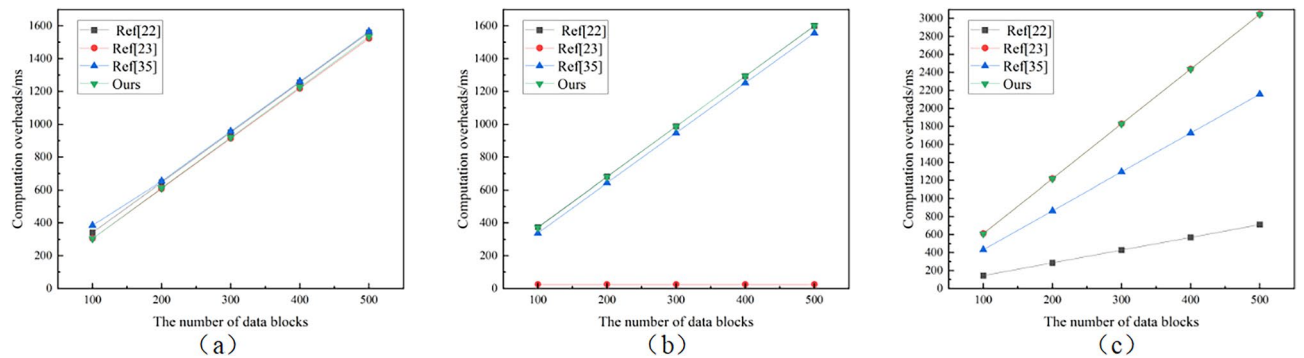
Schemes	Commitment technology	Fine-grained access control	Policy verification	AB-PRE	Dynamic management
Scheme ²²	-	✓	×	×	✓
Scheme ²³	Vector commitment	✓	×	×	✓
Scheme ³⁵	-	✓	×	×	✓
Ours scheme	Vector commitment	✓	✓	✓	✓

Table 2. Function comparison.

	Ref. ²²	Ref. ²³	Ref. ^{3,5}	Ours scheme
①	–	$dM_{G_1} + (2 + d)E_{G_1}$	–	$(2d - 1)M_{G_1} + 2dE_{G_1} + (d + 1)H_{Z_p}$
②	$(d + 11)M_{G_1} + (13 + d)E_{G_1} + 4A_{G_1} + 6H_{Z_p}$	$(d - 1)M_{G_1} + (1 + d)E_{G_1}$	$2P + (d + 4)M_{G_1} + (d + 11)E_{G_1} + 4H_{Z_p}$	$(d - 1)M_{G_1} + (d - 1)E_{G_1} + (d - 1)H_{Z_p}$
③	$4P + (d + 9)M_{G_1} + (d + 9)E_{G_1} + (d - 1)A_{G_1}$	$2P + M_{G_1} + 2E_{G_1}$	$3P + (d + 2)M_{G_1} + dE_{G_1}$	$6P + (d + 2)M_{G_1} + dE_{G_1} + dH_{Z_p}$
④	$dM_{G_1} + E_{G_1}$	$(2d + 1)M_{G_1} + (2d + 1)E_{G_1}$	$(3d + 1)M_{G_1} + (3d + 3)A_{G_1}$	$(2d - 1)M_{G_1} + (2d + 1)E_{G_1} + H_{Z_p}$

Table 3. Comparative analysis of computational overhead. Note: In the above table, we use ① to indicate “commitment generation,” ② to indicate “proof generation,” ③ to indicate “proof verification,” and ④ to indicate “privilege update and verification.”

Scheme	Date upload	Data sharing	Update phase
Ref. ²²	$(3d + 7) G_1 + d Z_p^* + d H $	$d(3 G_1 + Z_p^* + CT)$	$(d + 3) G_1 + (d + 1) Z_p^* + (2d + 1) H $
Ref. ²³	$(4d + 1) G_1 + Z_p^* + CT $	$(2d - 2) G_1 + 2d Z_p^* + 2d H $	$2d(G_1 + Z_p^*)$
Ref. ³⁵	$(d + 3) G_1 + d Z_p^* + CT + (d + 1) H $	$(2d + 3) G_1 + d Z_p^* $	$3d G_1 + 2 Z_p^* $
Ours scheme	$(d + 2) G_1 + 2 Z_p^* + CT + (d + 1) H $	$(d + 10) G_1 + 3 Z_p^* + (2d + 2) H $	$(2d + 3) G_1 + 4 Z_p^* + 5 H $

Table 4. Comparative analysis of communication overhead.**Figure 6.** Computational overhead required for different data blocks at various stages. (a) Proof generation (b) Proof validation (c) Commitment change management.

programming language is C, based on the bilinear paired cipher libraries (*PBCs*) and GNU multi-precision algorithm libraries (*GMPs*).

Proof generation phase computational overhead

Figure 6a illustrates the comparison results of the computational overhead in the proof generation phase. As the number of data blocks increases, the overheads of reference²², reference²³, reference³⁵, and our scheme exhibit a linear growth trend. Specifically, reference²² realizes the proof of time and attribute conditions by collaboratively constructing indexes and tokens between the data owner and the user. Reference²³ adopts an elliptic curve - based vector commitment scheme. Reference³⁵ requires the user to obtain a signature and a partial key through registration to form proofs of identity and attributes, while also demanding that the data owner generates the access policy ciphertexts embedded with attribute matching conditions. All of these are essentially processes of generating proofs to enable the verification of specific privileges or conditions. In contrast, our scheme innovatively combines the vector commitment mechanism with CP - ABE and naturally integrates the commitment proof process into the execution logic of the access policy, thus completing the commitment compliance checking simultaneously with attribute verification. The computational overhead of our scheme is significantly lower than that of reference²², reference²³, and reference³⁵, benefiting from the adoption of fewer exponential operations in the commitment generation phase and a more concise generation process.

Proving the computational overhead of the validation phase

The computational overhead of the proof verification stage comprises three parts: data access, commitment verification, and data validation. Reference²³ only addresses data validation, so it is excluded from the comparison. Reference²² completes proof verification by checking token-index matching (e.g., time-range overlap, attribute satisfaction of the access tree policy), and its process involves more index operations. In contrast, our scheme introduces hash operations, resulting in lower computational overhead than reference²². However, reference³⁵ verifies attribute-ciphertext matching by validating the user's signature and recovering the secret value via Lagrange interpolation. Due to fewer bilinear pairing operations, its computational overhead is slightly lower than ours but still within a reasonable range. The above conclusions are visualized in Fig. 6b.

Computational overhead of the dynamic change management phase of commitments

Figure 6c demonstrates the computational overhead comparison results of the permission change management phase (including the *ReEnc* algorithm and the *UpVer* algorithm). In a comprehensive comparison, the computational overhead of our scheme is within a reasonable range. Specifically, the update mechanism in reference²³ is the most efficient. It adopts the update computation technique based on version number design and avoids hash operations; however, its computational overhead is slightly higher than that of our scheme because it involves more multiplication and exponentiation operations. Reference²² realizes the dynamic revocation and restoration of privileges by updating the time component in the index and embedding the secret share of the revoked user after the data owner specifies the revocation list; since its scheme does not involve the key updating

session (and the related computational details are described in limited detail), its overhead is lower than that of our scheme. On the other hand, reference³⁵ utilizes CRT to update part of the ciphertext and realizes the dynamic management of attributes by manipulating the secret share of the revoking user; it is worth noting that the scheme of reference³⁵ does not include the *UpVer* algorithm phase, and its public description mainly focuses on the *ReEnc* algorithm phase. Therefore, only the overhead of its ciphertext updating part is calculated in the comparison, and its calculation result is also lower than our scheme.

Data-sharing phase communication overhead

In this section, we present the simulation results of our scheme alongside the data-sharing phase outcomes from reference²², reference²³ and reference³⁵, as illustrated in Fig. 7. Assume that the group $|\mathbb{G}_1|$ element size is 512 bits, the $|\mathbb{Z}_p^*|$ element size is 128 bits, and the hash is made with SHA - 256 (output 256 bits). We set the data blocks to be 100, 200, 300, 400, and 500 respectively. According to Fig. 7, the communication overhead increases gradually with the increase of data blocks. As reference²⁸, reference³² and reference³³ involve more group operations, the computation is complicated, which leads to higher communication overhead than the scheme in this article.

BFT comparison experiment

Authentication success rate experiment under different malicious nodes

Experimental setup: Set the total number of nodes $N = 100$. The proportion of malicious nodes f is configured as $f = 5\%$, $f = 10\%$, $f = 20\%$, $f = 30\%$, $f = 33\%$, and $f = 35\%$ to simulate attack scenarios under different scales of malicious node proportions. The comparison results of the authentication success rates under these varying malicious node proportions are presented in Fig. 8a.

Analysis: When the proportion of malicious nodes $f < 30\%$, as this proportion lies within the BFT fault tolerance threshold ($f \leq 33\%$, the system can resist interference through multiple rounds of consensus, and the authentication success rate remains above 90%. When $f = 33\%$, the Byzantine fault tolerance threshold of the system is $f = \frac{N-1}{3}$. If the number of malicious nodes exceeds this threshold, the system enters a critical state. Some authentication requests will time out due to network latency and consensus conflicts, causing the success rate to drop to around 61.2%, which corresponds to the inflection point in Fig. 8a where the success rate declines significantly in the 30% – 33% interval. When $f > 30\%$, the BFT threshold ($f = 33\%$) is breached. Malicious nodes gain control of the majority voting power, preventing the system from reaching an effective consensus and leading to a sharp decline in the authentication success rate. In particular, when $f = 35\%$, all authentication requests are rejected, and the success rate drops to 0%, which is consistent with the trend of the success rate approaching zero near 35% in Fig. 8a.

Performance metrics comparison experiments (percentage of malicious nodes: 0%, 5%, 10%, 15%, 20%, 25%, 30%)

Analysis: Delay increase: As the proportion of malicious nodes rises, malicious nodes may deliberately delay their responses or send invalid messages, triggering more consensus retransmission processes. Taking the data in Fig. 8b as an example, the average latency increases from approximately 123 ms to 248 ms when f ranges from 0% to 30%. Specifically, within the range of malicious node proportions shown in the figure, the latency increases by approximately 10–20 ms for every fixed-magnitude increase in the proportion. For instance, when $f = 30\%$, the average latency climbs to 248 ms due to the additional consensus validation rounds required as a result of malicious node interference. Throughput decreases: The throughput drops from 500 TPS (Transactions Per Second) when the proportion of malicious nodes is 0% to 250 TPS when this proportion reaches 30%, as more computational resources are required to handle malicious interference, which in turn exacerbates network congestion.

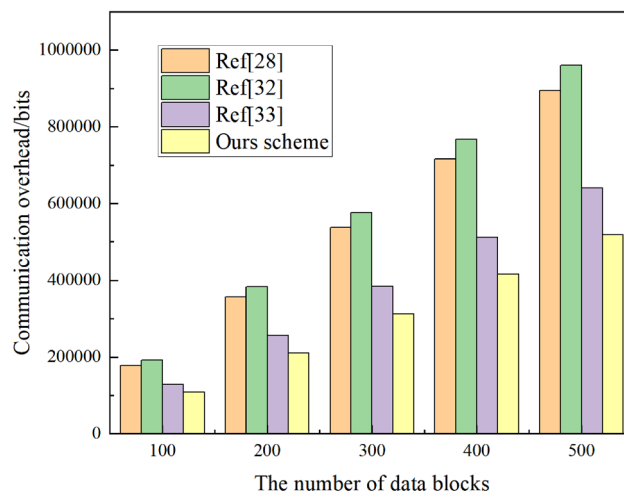


Figure 7. Communication overhead of different data blocks in the data-sharing phase.

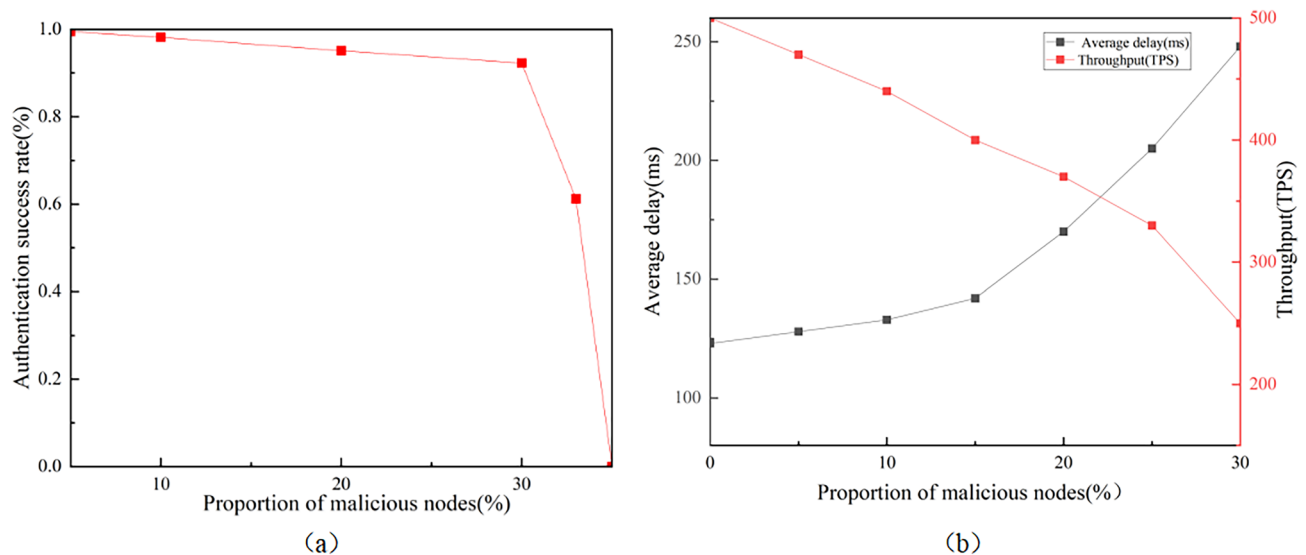


Figure 8. Comparison of authentication success rates and performance. **(a)** Comparison of authentication success rates. **(b)** Performance comparison.

Experiments show that as f approaches the BFT threshold (approximately 30% as inferred from the figure's trend, which aligns with the critical range of typical BFT systems), the authentication success rate decreases significantly. Meanwhile, performance metrics (latency and throughput) exhibit a linear degradation trend with the increase in f .

Conclusion

We propose a blockchain-based verifiable CP-ABE healthcare data privacy protection scheme for data sharing and dynamic updating scenarios in various departments within a hospital. The solution introduces blockchain to upload metadata on the chain, which reduces the need for trust between users and cloud service providers and is suitable for decentralized and distrusted scenarios. The scheme mainly verifies the reliability of the access policy with the help of a vector commitment mechanism to further guarantee data security; it applies the AB-PRE algorithm to re-encrypt data and performs key updates, thereby facilitating dynamic user data updates and ensuring valid data access. In addition, the solution can resist various attacks, such as replay attacks, which effectively improves system security. This scheme, verified by theoretical analysis and simulation experiments, is practicable and applicable to users with certain computing abilities. In the validation phase, the scheme shows high efficiency and meets the user's demand for immediate feedback on data integrity detection. However, the current scheme faces key challenges in cross-organizational healthcare data collaboration scenarios, mainly reflected in the efficiency constraints of dynamic policy updating. Future research will focus on constructing a fine-grained policy update framework and introducing an adaptive weighting mechanism to optimize the key update process during attribute modifications, thereby enhancing the scheme's security and practicality in complex distributed environments.

Data availability

The datasets generated and/or analysed during the current study are not publicly available due to ongoing related research projects. However, they are available from the corresponding author upon reasonable request

Received: 8 May 2025; Accepted: 21 July 2025

Published online: 27 July 2025

References

- Wang, M. et al. Medshare: A privacy-preserving medical data sharing system by using blockchain. *IEEE Trans. Serv. Comput.* **16**(1), 438–451 (2021).
- Afzal, M. K. et al. Unlocking 5g spectrum potential for intelligent iot: Opportunities, challenges, and solutions. *IEEE Commun. Mag.* **56**(10), 92–93 (2018).
- Ji, B. et al. A survey of computational intelligence for 6g: Key technologies, applications and trends. *IEEE Trans. Industr. Inf.* **17**(10), 7145–7154 (2021).
- Zhou, X., Liang, W., Kevin, I., Wang, K. & Yang, L. T. Deep correlation mining based on hierarchical hybrid networks for heterogeneous big data recommendations. *IEEE Trans. Comput. Soc. Syst.* **8**(1), 171–178 (2020).
- Zhou, X. et al. Distribution bias aware collaborative generative adversarial network for imbalanced deep learning in industrial iot. *IEEE Trans. Industr. Inf.* **19**(1), 570–580 (2022).
- Khafa, F., Feng, J., Zhang, Y., Chen, X. & Li, J. Privacy-aware attribute-based phr sharing with user accountability in cloud computing. *J. Supercomput.* **71**, 1607–1619 (2015).
- Miao, Y. et al. Secure online/offline data sharing framework for cloud-assisted industrial internet of things. *IEEE Internet Things J.* **6**(5), 8681–8691 (2019).

8. Qi, S., Lu, Y., Wei, W. & Chen, X. Efficient data access control with fine-grained data protection in cloud-assisted iiot. *IEEE Internet Things J.* **8**(4), 2886–2899 (2020).
9. Hao, J. et al. Secure data sharing with flexible user access privilege update in cloud-assisted iomt. *IEEE Trans. Emerg. Top. Comput.* **10**(2), 933–947 (2021).
10. Guo, R., Yang, G., Shi, H., Zhang, Y. & Zheng, D. O 3-r-cp-abe: An efficient and revocable attribute-based encryption scheme in the cloud-assisted iomt system. *IEEE Internet Things J.* **8**(11), 8949–8963 (2021).
11. Guo, R., Shi, H., Zhao, Q. & Zheng, D. Secure attribute-based signature scheme with multiple authorities for blockchain in electronic health records systems. *IEEE Access* **6**, 11 676–11 686 (2018).
12. Bramm, G., Gall, M. & Schütte, J. Blockchain-based distributed attribute based encryption. In *Proceedings of the 15th International Joint Conference on e-Business and Telecommunications (ICETE 2018)- Volume*, vol. 2, pp. 99–110 (2018).
13. Li, Q., Xia, B., Huang, H., Zhang, Y. & Zhang, T. Trac: Traceable and revocable access control scheme for mhealth in 5g-enabled iiot. *IEEE Trans. Industr. Inf.* **18**(5), 3437–3448 (2021).
14. Ren, Z., Yan, E., Chen, T. & Yu, Y. Blockchain-based cp-abe data sharing and privacy-preserving scheme using distributed kms and zero-knowledge proof. *J. King Saud Univ. Comput. Inf. Sci.* **36**(3), 101969 (2024).
15. Yang, H. & Yang, B. A blockchain-based approach to the secure sharing of healthcare data. In *Proceedings of the norwegian information security conference*. Nisk J Oslo, Norway, pp. 100–111 (2017).
16. Guo, H., Li, W., Nejad, M. & Shen, C.-C. Access control for electronic health records with hybrid blockchain-edge architecture. In *2019 IEEE international conference on blockchain (Blockchain)*. IEEE, 2019, 44–51.
17. Dagher, G. G., Mohler, J., Milojkovic, M. & Marella, P. B. Ancile: Privacy-preserving framework for access control and interoperability of electronic health records using blockchain technology. *Sustain. Cities Soc.* **39**, 283–297 (2018).
18. Yu, G. et al. Enabling attribute revocation for fine-grained access control in blockchain-iot systems. *IEEE Trans. Eng. Manage.* **67**(4), 1213–1230 (2020).
19. Sethi, K., Pradhan, A. & Bera, P. Pmter-abe: a practical multi-authority cp-abe with traceability, revocation and outsourcing decryption for secure access control in cloud systems. *Clust. Comput.* **24**, 1525–1550 (2021).
20. Yang, X., Li, T., Xi, W., Chen, A. & Wang, C. A blockchain-assisted verifiable outsourced attribute-based signcryption scheme for ehrs sharing in the cloud. *IEEE Access* **8**, 170 713–170 731 (2020).
21. Liu, P., Wang, J., Ma, H. & Nie, H. Efficient verifiable public key encryption with keyword search based on kp-abe. In *2014 Ninth International Conference on Broadband and Wireless Computing, Communication and Applications*. IEEE, 2014, pp. 584–589.
22. Miao, Y. et al. Time-controllable keyword search scheme with efficient revocation in mobile e-health cloud. *IEEE Trans. Mob. Comput.* **23**(5), 3650–3665 (2023).
23. Chu, S., Jiang, W. & Zhang, H. Blockchain transaction verification scheme based on vector commitments. *Int. J. High Speed Electron. Syst.* 2540411 (2025).
24. Srinivasan, S., Chepur, A., Papamanthou, C., Tomescu, A. & Zhang, Y. Hyperproofs: Aggregating and maintaining proofs in vector commitments. In *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3001–3018.
25. Campanelli, M., Fiore, D., Greco, N., Kolonelos, D. & Nizzardo, L. Incrementally aggregatable vector commitments and applications to verifiable decentralized storage. In *Advances in Cryptology—ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II 26*. Springer, 2020, pp. 3–35.
26. Tomescu, A., Abraham, I., Buterin, V., Drake, J., Feist, D. & Khovratovich, D. Aggregatable subvector commitments for stateless cryptocurrencies. In *International Conference on Security and Cryptography for Networks*. Springer, 2020, pp. 45–64.
27. Sánchez, D.C. Raziel: Private and verifiable smart contracts on blockchains. *arXiv preprint, arXiv:1807.09484*, (2018).
28. Doshi, N. An enhanced approach for cp-abe with proxy re-encryption in iot paradigm. *Jordanian J. Comput. Inf. Technol.* **8**(3), (2022).
29. Xiang, G., Li, B., Fu, X., Xia, M. & Ke, W. An attribute revocable cp-abe scheme. In *2019 Seventh International Conference on Advanced Cloud and Big Data (CBD)*. IEEE, pp. 198–203 (2019).
30. Edemacu, K., Jang, B. & Kim, J. W. Collaborative ehealth privacy and security: An access control with attribute revocation based on obdd access structure. *IEEE J. Biomed. Health Inform.* **24**(10), 2960–2972 (2020).
31. Yang, Y., Liu, X., Deng, R. H. & Li, Y. Lightweight sharable and traceable secure mobile health system. *IEEE Trans. Depend. Secure Comput.* **17**(1), 78–91 (2017).
32. Sun, J. et al. Privacy-preserving bilateral fine-grained access control for cloud-enabled industrial iot healthcare. *IEEE Trans. Industr. Inf.* **18**(9), 6483–6493 (2021).
33. Xu, S., Ning, J., Huang, X., Li, Y. & Xu, G. Untouchable once revoking: A practical and secure dynamic ehr sharing system via cloud. *IEEE Trans. Depend. Secure Comput.* **19**(6), 3759–3773 (2021).
34. Miao, Y. et al. Efficient privacy-preserving spatial data query in cloud computing. *IEEE Trans. Knowl. Data Eng.* **36**(1), 122–136 (2023).
35. Liang, X., Liu, Y. & Ning, J. An access control scheme with privacy-preserving authentication and flexible revocation for smart healthcare. *IEEE J. Biomed. Health Inf.* (2024).
36. Xu, J. et al. An identity management and authentication scheme based on redactable blockchain for mobile networks. *IEEE Trans. Veh. Technol.* **69**(6), 6688–6698 (2020).
37. Liu, J., Fan, Y., Sun, R., Liu, L., Wu, C. & Mumtaz, S. Blockchain-aided privacy-preserving medical data sharing scheme for e-healthcare system. *IEEE Internet Things J.* **10**(24), 21 377–21 388 (2023).
38. Catalano, D. & Fiore, D. Vector commitments and their applications. In *Public-Key Cryptography—PKC 2013: 16th International Conference on Practice and Theory in Public-Key Cryptography, Nara, Japan, February 26–March 1, 2013. Proceedings 16*. Springer, 2013, pp. 55–72.

Acknowledgements

I sincerely thank my supervisor for her meticulous guidance during the thesis writing and the senior lab members for their warm assistance.

Author contributions

Z.L.I.: Conceptualization, Methodology, Software, Investigation, Formal analysis, Validation; Writing—original draft, Writing review and editing ; D.G.f: Supervision, Methodology, Formal analysis, Validation , Investigation, Visualization, Validation. Y.H: Conceptualization, Formal analysis, Resources. All authors reviewed the manuscript.

Funding

This work was supported in part by the National Natural Science Foundation of China under Grant 61662089. Yunnan Minzu University Graduate Research Innovation Fund Project (2023SKY007).

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to G.D.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025